MACHINE LEARNING

Neural networks and language models from first principles

UNIT 2

Language Models

Tokenisation, next-token prediction, attention, the transformer, pretraining, scaling laws, decoding and interpretability

2.1 Tokenisation

2.2 Next-token prediction

2.3 Attention

2.4 The transformer

2.5 Pretraining

2.6 Scaling laws

2.7 Decoding

2.8 Inside the model

First edition, October 2026

Contents

About this unit . . . 3
Before you start . . . 3
What this unit needs . . . 3
2.1 Tokenisation . . . 4
Core ideas . . . 4
Reading . . . 11
Labs . . . 11
Problem set . . . 12
2.2 Next-token prediction . . . 14
Core ideas . . . 14
Reading . . . 18
Labs . . . 18
Problem set . . . 20
2.3 Attention . . . 21
Core ideas . . . 21
Reading . . . 25
Labs . . . 25
Problem set . . . 26
2.4 The transformer . . . 28
Core ideas . . . 28
Reading . . . 35
Labs . . . 35
Problem set . . . 36
2.5 Pretraining . . . 37
Core ideas . . . 37
Reading . . . 43
Labs . . . 44
Problem set . . . 45
2.6 Scaling laws . . . 46
Core ideas . . . 46
Reading . . . 50
Labs . . . 50
Problem set . . . 51
2.7 Decoding . . . 52
Core ideas . . . 52
Reading . . . 57
Labs . . . 58
Problem set . . . 59
2.8 Inside the model . . . 60
Core ideas . . . 60
Reading . . . 66
Labs . . . 67
Problem set . . . 68
Sign-off . . . 69

Machine Learning, First edition, October 2026. © 2026 O'Neill Group LLC. PyTorch is a trademark of the Linux Foundation. Apple, Apple Silicon and Metal are trademarks of Apple Inc. NVIDIA and CUDA are trademarks of NVIDIA Corporation. Vulkan is a registered trademark of the Khronos Group Inc.

Hugging Face is a trademark of Hugging Face, Inc. All other trademarks belong to their owners. The models and datasets the course uses keep their own licences, listed in the Handbook.

UNIT 2

About this unit

Unit 2 builds a language model from an empty file and then studies it. It starts with the tokeniser that turns text into numbers, and with the simplest language models, counts of what follows what, which set the baseline. It derives attention, assembles the transformer that every modern language model uses, and pretrains one on 300 million tokens of children's stories in an afternoon on the M2 Pro. It then measures how loss scales with model size and data, compares the ways of turning probabilities into text, and looks inside the trained model to find the mechanisms it learned. From this unit on, programs use PyTorch, and the model, the tokeniser and the training loop live in `aif`, where Units 3 and 4 reuse them.

Chapter	You build	Hours
2.1	A byte-level BPE tokeniser that matches SmolLM2's, and its compression measured	5
2.2	N-gram models with Kneser–Ney smoothing, and a neural language model that beats them	6
2.3	Attention in NumPy and in `tiny`, checked against PyTorch, and a model that learns to copy	6
2.4	A transformer with RoPE, GQA and SwiGLU, its parameters and FLOPs counted and checked	6
2.5	A data pipeline, a training loop, and a 28M-parameter model pretrained on TinyStories	8
2.6	A scaling-law sweep and a prediction of a larger run's loss, checked	6
2.7	Greedy, beam, temperature, top-k, top-p and min-p decoding, and the KV cache	6
2.8	Induction heads found and switched off, the logit lens, and a probe for dialogue	7

Before you start

Finish Unit 1. Run `uv run -m u2.prepare` early: it downloads 2.2 GB of stories and encodes them, which takes a few minutes.

What this unit needs

A GPU makes the unit comfortable. Chapter 2.5's run takes just under five hours on the M2 Pro, and Chapter 2.6's sweep about three. On a CPU, use the `small` preset and fewer tokens, and expect weaker stories.

CHAPTER 2.1

Tokenisation

TIME	5 hours
YOU BUILD	A byte-level BPE tokeniser, trained on all 2.2 GB of TinyStories in under a minute and checked against a naive implementation; the same encoder loading SmolLM2's vocabulary and producing exactly the tokens Hugging Face's library does; compression measured against vocabulary size on stories and on code; and the failures that tokens cause, demonstrated.
NEEDS	Unit 1. Run `uv run -m u2.prepare` once to download TinyStories.

WHY IT MATTERS

A language model sees only a sequence of integers, *tokens*, chosen by a tokeniser that is fixed before training begins and cannot change afterwards. The tokeniser decides how long every sequence is, and so how much every step of training and generation costs. It determines which strings the model can treat as units and which it must assemble from fragments, which is why fluent models make mistakes in spelling and arithmetic. It also affects how well the model handles text unlike its training data. This chapter builds the tokeniser that every model in Unit 2 uses, and shows that it is equivalent to the one in Unit 3.

SKIP TEST

1. Why do modern tokenisers start from bytes rather than characters or words?
2. Given the corpus "low low lower lowest", what is the first merge byte-pair encoding makes, and why?
3. Why do many recent tokenisers split numbers into single digits?
4. A tokeniser trained on stories encodes Python code at two bytes per token. Why, and what does that cost a model?

If all four are easy, read the sections on training efficiently and on failures, and do Labs 2.1.3 and 2.1.4.

Core ideas

From text to integers

A model needs a finite vocabulary of symbols, each with its own embedding vector. There are three obvious choices. *Words* give short sequences but an unbounded vocabulary: every name, typo and new word would need an entry, and those missing from the vocabulary could not be represented at all. *Characters* give a manageable vocabulary for one alphabet but long sequences, and Unicode has over 150,000 of them. *Bytes* give a vocabulary of exactly 256, can represent any text in UTF-8, and make sequences long: one byte per letter in English, two or three per character in most other scripts.

Subword tokenisation is a compromise between these. Frequent strings, such as common words and common pieces of words, get single tokens, and rare strings are spelled out from smaller pieces, down to single bytes if need be. The vocabulary stays fixed, typically between 8,000 and 250,000 tokens, every input can be encoded, and common text is short.

Byte-pair encoding

Byte-pair encoding (BPE) learns the subwords from a corpus. Start with the corpus as bytes and a vocabulary of the 256 byte values. Count every adjacent pair of symbols; replace the most frequent pair everywhere

with a new symbol, and add it to the vocabulary; repeat until the vocabulary has the size you want. Each step records a *merge*, a rule that two symbols combine into a third, and the merges, in the order they were learned, are the tokeniser.

Merges never cross the boundaries of *chunks*, the pieces a regular expression splits text into before BPE sees it. GPT-2's expression, which this course uses, keeps a word together with the space before it, separates punctuation and runs of whitespace, and recognises English contractions. SmolLM2 and many recent tokenisers also split every digit into its own chunk:

GPT-2's pre-tokenisation pattern, and a pattern that isolates digits — code/aif/bpe.py

```
GPT2_PATTERN = r"""'(?:[sdmt]|ll|ve|re)| ?\p{L}+| ?\p{N}+| ?[^\s\p{L}\p{N}]+|\s+(?!\S)|\s+"""
DIGIT = regex.compile(r"(\p{N})")
```

Splitting text into chunks, with digits isolated — code/aif/bpe.py

```
def chunks(text: str, pattern: regex.Pattern, split_digits: bool) -> list[str]:
    if not split_digits:
        return pattern.findall(text)
    out = []
    for piece in DIGIT.split(text):
        if piece:
            out.extend(pattern.findall(piece))
    return out
```

Training efficiently

The naive algorithm recounts every pair in the corpus after each merge, which is fast enough for a page of text and far too slow for 2.2 GB. Two observations make it fast. First, merges never cross chunks, so the corpus reduces to its distinct chunks and their counts: the 530 million chunks of TinyStories are only 59,732 distinct strings, mostly words. Second, a merge only changes pair counts in the words that contain it, so the counts can be updated in place, and a heap finds the most frequent pair without scanning them all.

Counting chunks, the only pass over the full corpus — code/aif/bpe.py

```
def count_chunks(
    texts, pattern: str = GPT2_PATTERN, split_digits: bool = True, special: list[str] = ()
) -> Counter:
    compiled = regex.compile(pattern)
    splitter = regex.compile("|".join(regex.escape(s) for s in special)) if special else None
    counts = Counter()
    for text in texts:
        for part in splitter.split(text) if splitter else [text]:
            counts.update(chunks(part, compiled, split_digits))
    return counts
```

Replacing every occurrence of a pair in one word — code/aif/bpe.py

```
def merge_pair(ids: list[int], pair: tuple[int, int], new: int) -> list[int]:
    out, i = [], 0
    while i < len(ids):
        if i + 1 < len(ids) and ids[i] == pair[0] and ids[i + 1] == pair[1]:
            out.append(new)
            i += 2
        else:
            out.append(ids[i])
            i += 1
    return out
```

Training: pair counts kept up to date, a heap to find the most frequent pair, stale heap entries skipped — code/aif/bpe.py

```
def train(
    counts: Counter,
    vocab_size: int,
    special: list[str] = (),
    pattern: str = GPT2_PATTERN,
```

```
    split_digits: bool = True,
) -> BPE:
    words = [list(chunk.encode("utf-8")) for chunk in counts]
    freq = list(counts.values())
    pairs = Counter()
    where = defaultdict(set)
    for i, w in enumerate(words):
        for p in zip(w, w[1:]):
            pairs[p] += freq[i]
            where[p].add(i)
    heap = [(-n, p) for p, n in pairs.items()]
    heapify(heap)

    merges = []
    while len(merges) < vocab_size - 256 - len(special) and heap:
        n, best = heappop(heap)
        if pairs.get(best, 0) != -n:
            continue
        new = 256 + len(merges)
        merges.append(best)
        changed = set()
        for i in where.pop(best):
            w = words[i]
            merged = merge_pair(w, best, new)
            if len(merged) == len(w):
                continue
            for p in zip(w, w[1:]):
                pairs[p] -= freq[i]
                changed.add(p)
            for p in zip(merged, merged[1:]):
                pairs[p] += freq[i]
                where[p].add(i)
                changed.add(p)
            words[i] = merged
        for p in changed:
            if pairs[p] > 0:
                heappush(heap, (-pairs[p], p))
            else:
                del pairs[p]
    return from_merges(merges, list(special), pattern, split_digits)
```

`aif.corpus` counts chunks in parallel across processes, splitting the file at document boundaries, and `u2.prepare` trains the course's 8,192-token vocabulary and encodes the whole corpus to a file of 16-bit token ids:

Training the tokeniser and encoding the corpus — code/u2/prepare.py

```
if not tokenizer.exists():
    start = time.perf_counter()
    counts = corpus.count_file(train_txt)
    tok = bpe.train(counts, args.vocab, special=[corpus.EOT])
    tok.save(tokenizer)
    print(
        f"tokeniser: {len(counts):,} distinct chunks, {args.vocab} tokens "
        f"in {time.perf_counter() - start:.0f} s"
    )
tok = bpe.BPE.load(tokenizer)

stats = {}
for split, path in (("train", train_txt), ("valid", valid_txt)):
    start = time.perf_counter()
    n = corpus.encode_file(path, tokenizer, out / f"{split}-{args.vocab}.bin")
    size = path.stat().st_size
    stats[split] = {"tokens": n, "bytes": size}
    print(
        f"{split}: {size / 1e6:,.0f} MB of text -> {n / 1e6:,.1f}M tokens "
        f"({size / n:.2f} bytes per token) in {time.perf_counter() - start:.0f} s"
    )
```

All of TinyStories, tokenised, on the M2 Pro's twelve cores

```
$ uv run -m u2.prepare
tokeniser: 59,732 distinct chunks, 8192 tokens in 23 s
train: 2,228 MB of text → 536.2M tokens (4.15 bytes per token) in 25 s
valid: 23 MB of text → 5.4M tokens (4.16 bytes per token) in 2 s
the first complete validation story begins: 'Once upon a time, in a warm and sunny place,
    there was a big pit. A little boy named Tom liked to play near the pit.'
```

`u2/bpe_check.py` checks the fast trainer against the plain algorithm. It trains the first 200 merges both ways, on three million characters, and checks that every merge agrees. It then checks that every text decodes back to itself, and that the encoder below reproduces a production tokeniser exactly:

The textbook algorithm, recounting everything after each merge — code/u2/bpe_check.py

```python
def naive_merges(counts: Counter, merges: int) -> list[tuple[int, int]]:
    words = {tuple(chunk.encode("utf-8")): n for chunk, n in counts.items()}
    out = []
    for new in range(256, 256 + merges):
        pairs = Counter()
        for w, n in words.items():
            for p in zip(w, w[1:]):
                pairs[p] += n
        best = min(pairs, key=lambda p: (-pairs[p], p))
        out.append(best)
        words = {tuple(bpe.merge_pair(list(w), best, new)): n for w, n in words.items()}
    return out
```

The fast trainer, the round trip and SmolLM2's tokeniser, checked

```
$ uv run -m u2.bpe_check
the first 200 merges of the fast trainer equal the naive trainer's
the first ten: 'he', ' t', ' a', ' s', ' w', 'nd', ' the', 'ed', ' b', ' to'
40 texts, from stories to random Unicode, decode to exactly themselves
SmolLM2's vocabulary loaded into aif.bpe: identical tokens to Hugging Face's on 47 of 47
    texts
SmolLM2's vocabulary has no token for 21 byte values, which both encoders drop: 0x4, 0x6,
    0x13, 0x14, 0x16, 0x1d, ...
```

The first merges are the most common pairs in English children's stories: "he", then " t", " a", " s", " w", "nd", and by the seventh merge " the" is a single token.

Encoding

To encode a chunk, start from its bytes and apply merges in the order they were learned: among the adjacent pairs that have a merge, pick the one learned earliest, merge every occurrence, and repeat until no pair has a merge. Applying them in this order, rather than greedily by length, reproduces exactly the segmentation that training produced.

Encoding one chunk: always the earliest-learned merge first — code/aif/bpe.py

```python
def encode_chunk(self, data: bytes) -> list[int]:
    ids = [self.byte_id[b] for b in data if self.byte_id[b] is not None]
    while len(ids) > 1:
        pairs = set(zip(ids, ids[1:]))
        best = min(pairs, key=lambda p: self.rank.get(p, len(self.rank)))
        if best not in self.rank:
            break
        ids = merge_pair(ids, best, self.merged[best])
    return ids
```

Encoding text: special tokens, chunks, a cache, and decoding — code/aif/bpe.py

```
def encode(self, text: str) → list[int]:
    out = []
    parts = self.special_split.split(text) if self.special_split else [text]
    for part in parts:
        if part in self.special:
            out.append(self.special[part])
            continue
        for chunk in chunks(part, self.pattern, self.split_digits):
            ids = self.cache.get(chunk)
            if ids is None:
                ids = self.encode_chunk(chunk.encode("utf-8"))
                self.cache[chunk] = ids
            out.extend(ids)
    return out

def decode(self, ids: list[int]) → str:
    return b"".join(self.token_bytes[i] for i in ids).decode("utf-8", errors="replace")
```

The cache matters because most chunks are common words seen thousands of times, so after the first few megabytes almost every chunk is a dictionary lookup. *Special tokens*, such as `◁endoftext▷` which separates documents, are matched before chunking and never produced by merges. Because they are matched in the input, text that contains the string `◁endoftext▷` is encoded as the special token itself, as it is by default in Hugging Face's tokenisers. A system that places untrusted text in a prompt, such as a web page, should strip or escape special-token strings first, or a document could end the conversation it was placed in.

Loading someone else's tokeniser

Hugging Face's `tokenizer.json` stores a byte-level BPE vocabulary as strings, mapping each byte to a printable character so that tokens are readable in the file. GPT-2's table maps the printable ASCII and Latin-1 bytes to themselves and the rest to characters from U+0100 onwards, so a space is stored as "Ġ".

GPT-2's byte-to-character table — code/aif/bpe.py

```
def gpt2_byte_map() → dict[int, str]:
    """The byte-to-character table GPT-2 uses to store token bytes as printable text."""
    printable = (
        list(range(ord("!"), ord("~") + 1))
        + list(range(ord("¡"), ord("¬") + 1))
        + list(range(ord("®"), ord("ÿ") + 1))
    )
    table, extra = {}, 0
    for b in range(256):
        if b in printable:
            table[b] = chr(b)
        else:
            table[b] = chr(256 + extra)
            extra += 1
    return table
```

With the table inverted, `BPE.from_huggingface` reads SmolLM2's 49,152 tokens and 48,900 merges into the same class, and the check above shows it producing exactly the tokens of the reference implementation on stories, code, chat markup and random Unicode. The check also found that SmolLM2's vocabulary has no token for 21 of the 256 byte values, mostly control characters and bytes that never occur in valid UTF-8, and that the reference implementation silently drops them. `aif.bpe` does the same, and Unit 3 relies on the two agreeing.

Compression

How many bytes a token covers on average measures how well a tokeniser compresses text, and so how long the sequences a model must process are. `u2/bpe_train.py` trains vocabularies from 512 to 16,384 tokens on TinyStories and measures each on its own domain, stories, and on another, the source code of PyTorch's neural-network modules:

One count of the corpus, then a vocabulary of each size — code/u2/bpe_train.py

```
start = time.perf_counter()
counts = corpus.count_file(data.tinystories("train"))
print(
    f"counted {sum(counts.values()):,} chunks ({len(counts):,} distinct) in "
    f"{time.perf_counter() - start:.0f} s"
)
rows = []
for size in args.sizes:
    start = time.perf_counter()
    tok = bpe.train(counts, size, special=[corpus.EOT])
    seconds = time.perf_counter() - start
    per_story = len(stories.encode()) / len(tok.encode(stories))
    per_code = len(code.encode()) / len(tok.encode(code))
    rows.append(
        {"vocab": size, "stories": per_story, "code": per_code, "seconds": seconds}
    )
    print(
        f"vocabulary {size:6d}: {per_story:5.2f} bytes per token on stories, "
        f"{per_code:5.2f} on Python "
        f"code; trained in {seconds:4.1f} s"
    )
```

Bytes per token against vocabulary size

```
$ uv run -m u2.bpe_train
counted 530,415,674 chunks (59,732 distinct) in 23 s
vocabulary    512:  2.43 bytes per token on stories,  1.33 on Python code; trained in  0.8 s
vocabulary   1024:  3.19 bytes per token on stories,  1.45 on Python code; trained in  1.1 s
vocabulary   2048:  3.69 bytes per token on stories,  1.52 on Python code; trained in  1.4 s
vocabulary   4096:  3.98 bytes per token on stories,  1.60 on Python code; trained in  1.6 s
vocabulary   8192:  4.10 bytes per token on stories,  1.99 on Python code; trained in  1.8 s
vocabulary  16384:  4.12 bytes per token on stories,  2.26 on Python code; trained in  1.9 s
SmolLM2 (49,152 tokens, trained on web text and code): 4.06 on stories, 3.77 on code
```

On stories, compression rises quickly and then saturates near 4.1 bytes per token, because TinyStories has only 60,000 distinct words and by 8,192 tokens nearly every common word has its own. On code the same vocabularies manage 1.3 to 2.3 bytes per token, because merges learned from stories rarely occur in code. SmolLM2's tokeniser, trained on web text and code, compresses stories as well as the specialised one and code almost twice as well. The domain of the training corpus decides what a tokeniser compresses well, more than the vocabulary's size does. A poorly matched tokeniser makes every sequence longer, which costs more steps per document and more memory for attention.

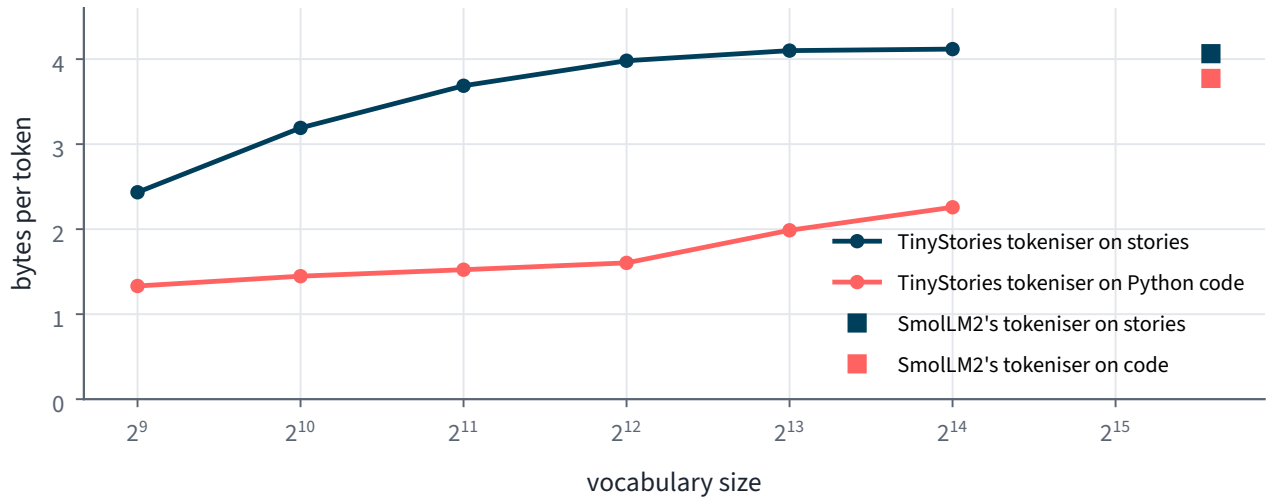


Figure 2.1.1. Compression against vocabulary size. On its own domain the tokeniser saturates once most words have tokens of their own. On code its compression stays low at every size.

What tokens hide

A model that sees tokens does not see the letters inside them directly. `u2/tokens.py` shows both tokenisers' pieces for a few strings, with spaces drawn as dots:

Tokens as text, with undecodable fragments shown as hex bytes — code/u2/tokens.py

```
def pieces(tok: bpe.BPE, text: str) -> str:
    out = []
    for i in tok.encode(text):
        raw = tok.token_bytes[i]
        try:
            out.append(raw.decode("utf-8").replace("\n", "\\n").replace(" ", "·"))
        except UnicodeDecodeError:
            out.append("<" + raw.hex() + ">")
    return "|".join(out)
```

How two tokenisers split ten strings

```
$ uv run -m u2.tokens
'Once upon a time'
  TinyStories 8,192:  4 tokens  Once|·upon|·a|·time
  SmolLM2 49,152:     4 tokens  Once|·upon|·a|·time
'Tom had 1234567 apples.'
  TinyStories 8,192: 12 tokens  Tom|·had|·|1|2|3|4|5|6|7|·apples|.
  SmolLM2 49,152:    12 tokens  Tom|·had|·|1|2|3|4|5|6|7|·apples|.
'hello Hello HELLO'
  TinyStories 8,192:  8 tokens  hello|·He|llo|·H|E|L|L|O
  SmolLM2 49,152:     5 tokens  hello|·Hello|·H|EL|LO
'strawberry'
  TinyStories 8,192:  4 tokens  st|ra|w|berry
  SmolLM2 49,152:     3 tokens  st|raw|berry
'Supercalifragilistic'
  TinyStories 8,192:  9 tokens  S|up|erc|al|if|rag|il|ist|ic
  SmolLM2 49,152:     6 tokens  Super|cal|if|rag|il|istic
'𠀀𠀁𠀂'
  TinyStories 8,192:  9 tokens  <e6>|<97>|<a5>|<e6>|<9c>|<ac>|<e8>|<aa>|<9e>
  SmolLM2 49,152:     6 tokens  <e697>|<a5>|𠀁|<e8>|<aa>|<9e>
'𠀀'
  TinyStories 8,192:  4 tokens  <f0>|<9f>|<99>|<82>
  SmolLM2 49,152:     2 tokens  <f09f>|<9982>
'def f(x):\n    return x'
  TinyStories 8,192: 11 tokens  de|f|·f|(|x|)|:|\n·|··|·return|·x
  SmolLM2 49,152:     8 tokens  def|·f|(|x|):|\n···|·return|·x
'the end. '
  TinyStories 8,192:  4 tokens  the|·end|.|·
  SmolLM2 49,152:     4 tokens  the|·end|.|·
'the end.'
  TinyStories 8,192:  3 tokens  the|·end|.
  SmolLM2 49,152:     3 tokens  the|·end|.
```

"hello", "Hello" and "HELLO" share nothing at the token level, so the model must learn their relationship from data. "strawberry" arrives as three or four pieces, `st|raw|berry` or `st|ra|w|berry`, so the model never sees its r's one at a time, which is why counting them is hard for a language model. Numbers become sequences of digits, which helps arithmetic, since every number is spelled the same way wherever it appears. Tokenisers that merge digit groups write 1234567 as arbitrary chunks such as "123" and "4567". The TinyStories tokeniser, which saw almost no Japanese and no emoji, breaks them into raw bytes, three tokens for each Japanese character and four for the emoji. SmolLM2's does better, keeping 𠀁 whole and spending two tokens on the emoji, but with both tokenisers the same sentence costs several times more tokens in Japanese than in English. Finally, a trailing space is its own token, so a prompt ending in a space puts the model in an unusual position, since in training text a space almost always arrives attached to the following word.

KEY IDEA

A tokeniser is a compression scheme learned from a corpus and fixed for the life of the model. It determines sequence lengths, and therefore cost. It also determines what the model can see directly, and the model must learn from data how the pieces it splits apart fit together.

IN THE LIBRARIES

Production tokenisers are written in fast languages: OpenAI's `tiktoken` and Hugging Face's `tokenizers` in Rust, Google's SentencePiece in C++. They implement the algorithms of this chapter, with SentencePiece also offering a probabilistic alternative, the unigram model. `tokenizers` reads and writes the `tokenizer.json` format that this chapter's loader understands, and `AutoTokenizer.from_pretrained` wraps it with a model's chat template.

Reading

- Rico Sennrich, Barry Haddow and Alexandra Birch, "Neural Machine Translation of Rare Words with Subword Units", ACL 2016: BPE for language.
- Alec Radford and others, "Language Models are Unsupervised Multitask Learners", OpenAI, 2019, section 2.2: byte-level BPE.
- Taku Kudo and John Richardson, "SentencePiece: A Simple and Language Independent Subword Tokenizer", EMNLP 2018.
- Aleksandar Petrov and others, "Language Model Tokenizers Introduce Unfairness Between Languages", NeurIPS 2023.

Labs

LAB 2.1.1 BPE by hand — 1 hour

Goal. Learn the algorithm by running it on paper.

Code. `code/aif/bpe.py`

1. Take the corpus "low low low lower lower newest newest newest widest", split into words with the space attached to the word that follows.
2. Perform the first six merges by hand, recording the pair counts before each.
3. Check your merges with `bpe.count_chunks` and `bpe.train`, then encode "lowest" by hand and with the tokeniser.

DONE WHEN

- [ ] Your six merges and the encoding of "lowest" match the program's.
- [ ] You can explain any tie and how the program breaks it.

EVIDENCE FOR THE NOTEBOOK

- The pair counts after each merge, and the program's output.

LAB 2.1.2 Choosing a vocabulary size — 1 hour

Goal. Weigh compression against the cost of a larger vocabulary.

Code. `code/u2/bpe_train.py`

1. For vocabularies of 1,024, 4,096 and 16,384, compute how many tokens TinyStories' validation set becomes.

2. For the Chapter 2.5 model, compute how the parameters of the embedding and the FLOPs of the output layer change with the vocabulary.
3. Estimate the total training FLOPs for the same number of *bytes* of text with each vocabulary.

DONE WHEN

- [ ] Your table has the tokens, the embedding parameters and the FLOPs per byte for each size.
- [ ] You can say which vocabulary minimises training cost per byte for this model, and why it is not the largest.

EVIDENCE FOR THE NOTEBOOK

- The table and the argument.

LAB 2.1.3 A tokeniser for code

1.5 hours

Goal. See how the training corpus shapes a tokeniser.

Code. `code/u2/bpe_train.py`

1. Train an 8,192-token tokeniser on the Python files of PyTorch's installation, using `bpe.count_chunks` over their text.
2. Measure its compression on code it did not see, such as the course's `aif` package, and on TinyStories, and compare with the story tokeniser and SmolLM2's.
3. Look at its 100 longest tokens and say what they are.

DONE WHEN

- [ ] You have a 3 by 2 table of compression for three tokenisers on two domains.
- [ ] You can describe what kinds of string your code tokeniser learned to keep whole.

EVIDENCE FOR THE NOTEBOOK

- The table and the list of long tokens.

LAB 2.1.4 Tokens and letters

1.5 hours

Goal. Measure how tokenisation hides characters.

Code. `code/u2/tokens.py`

1. For the 1,000 most common words in TinyStories, record how many tokens each takes in both tokenisers, and how many words are a single token.
2. For the same words with their first letter capitalised, and in capitals, repeat the count.
3. Encode one sentence translated into English, French, Greek and Japanese, and compare tokens per character.

DONE WHEN

- [ ] You have both counts, and the fraction of words whose capitalised forms share no token with their lower-case form.
- [ ] Your table shows the cost of the four languages in tokens per character.

EVIDENCE FOR THE NOTEBOOK

- The counts, the table, and a paragraph on what each implies for a model.

Problem set

1. Show that BPE training is deterministic given a tie-breaking rule, and give a corpus where two different tie-breaking rules produce different tokenisers.

2. Why must a tokeniser encode with the merges in the order they were learned, rather than taking the longest token that matches at each position? Give a string where the two disagree.
3. A model's context holds 2,048 tokens. With 4.1 bytes per token on English and 1.3 on Japanese, how many characters of each fit?
4. Why is it important that ordinary text can never produce the token `◁endoftext▷`? What could happen if it could?
5. SmolLM2's vocabulary lacks the byte 0x04. What does the model see if a document contains it? Is silently dropping it the right behaviour?

COMPLETION CHECKLIST

- [ ] I can explain byte-level BPE, train it by hand, and encode with its merges.
- [ ] I can explain pre-tokenisation and why digits are often split.
- [ ] I can load a Hugging Face tokeniser into my own encoder and show that they agree.
- [ ] I can measure compression and explain how vocabulary size and training domain affect it.
- [ ] Lab 2.1.1–2.1.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.2

Next-token prediction

TIME	6 hours
YOU BUILD	Unigram to 5-gram models with interpolated Kneser–Ney smoothing, counted from 100 million tokens in a few minutes; perplexity and bits per byte computed properly; samples that show what n-grams can and cannot do; and Bengio's neural language model, which beats the best of them with a context of eight tokens.
NEEDS	Chapter 2.1, and Chapter 1.3's cross-entropy.

WHY IT MATTERS

A language model is a probability distribution over the next token given the ones before it. The simplest such models count how often each token follows each context. They are the baseline every neural model must beat, and they make the measures of language modelling, perplexity and bits per byte, concrete. Their main weakness, that most contexts are never seen in training, is the problem that neural networks solve by learning which contexts are alike.

SKIP TEST

1. Write the probability of the sentence "the cat sat" as a product of conditional probabilities, and say what a bigram model assumes about it.
2. Why does a trigram model estimated by counting need smoothing, and what does Kneser–Ney smoothing do that adding one to every count does not?
3. A model's cross-entropy is 2.5 nats per token. What is its perplexity, and its bits per byte at 4.16 bytes per token?
4. Why compare models with different tokenisers in bits per byte instead of per token?

If all four are easy, read the section on Kneser–Ney and do Labs 2.2.3 and 2.2.4.

Core ideas

The chain rule

The probability of a sequence of tokens factorises exactly into conditional probabilities, one per token:

$$P(x_1, \ldots, x_n) = \prod_{t=1}^{n} P(x_t \mid x_1, \ldots, x_{t-1}).$$

A language model is any way of estimating those conditionals. Training one means maximising the probability of real text, which by Chapter 1.3 means minimising the cross-entropy of each token given its predecessors, averaged over the corpus. Generating text means sampling from the conditionals one token at a time, each sample becoming part of the next context.

Measuring a language model

The cross-entropy per token, in nats, is the basic measure. Its exponential is the *perplexity*: a model with perplexity 20 is, on average, as uncertain about the next token as a uniform choice among 20. Both depend on the tokeniser, because a tokeniser with longer tokens makes each token harder to predict and needs fewer of them. Dividing the cross-entropy by $\ln 2$ and by the average bytes per token gives *bits per byte*, a measure that compares models with any tokeniser on the same text. It is also the size an ideal compressor using the model would achieve: a model at 0.5 bits per byte could store TinyStories in a sixteenth of its size.

N-gram models

An *n-gram* model assumes that the next token depends only on the previous $n-1$: a *bigram* model looks back one token, a *trigram* model two. Its maximum-likelihood estimate is a ratio of counts, $P(w \mid h) = c(hw)/c(h)$, where h is the context and c counts occurrences in the training text. Most contexts of more than a few tokens never occur in training, and of those that do, most continuations were never seen after them. A raw count assigns those probability zero, and a single zero makes the probability of a whole test text zero and its cross-entropy infinite.

Smoothing moves probability from seen to unseen events. Adding one to every count is the simplest scheme and works badly, because it gives every unseen continuation half the weight of a continuation seen once. *Absolute discounting* subtracts a fixed amount D, typically 0.75, from every nonzero count and redistributes the total through a lower-order model:

$$P(w \mid h) = \frac{\max(c(hw) - D, 0)}{c(h)} + \frac{D \cdot N_{1+}(h\,\cdot)}{c(h)} \cdot P(w \mid h'),$$

where h' is the context shortened by one token and $N_{1+}(h\,\cdot)$ counts the distinct tokens seen after h, so that contexts followed by many different tokens give more weight to the lower order. *Kneser–Ney* smoothing changes what the lower orders count. In TinyStories "named" is frequent, but almost always after "girl" or "boy" in "a little girl named". A unigram model built from raw counts would predict "named" in any unfamiliar context. Kneser–Ney's lower orders count instead how many *different* contexts a token follows, its *continuation count*, so a token that is common only in one context gets little weight when that context is absent.

Counting at scale

Counting 5-grams with Python dictionaries over 100 million tokens would take hours and tens of gigabytes. Since the vocabulary has 8,192 tokens, 13 bits each, up to four tokens pack into one 64-bit integer, and NumPy can sort and count a hundred million of those in seconds. A 5-gram needs 65 bits, one too many, so every n-gram is kept as a pair: the integer for its context of up to four tokens, and the token that follows. Counting pairs takes two sorts, one to number the distinct contexts and one to count each context's continuations:

Runs of tokens as integers, and n-grams as counted (context, token) pairs — code/u2/ngram.py

```python
def gram_keys(ids: np.ndarray, n: int, vocab: int) -> np.ndarray:
    """Every run of n tokens as one integer; 13 bits a token, so n is at most 4."""
    keys = np.zeros(len(ids) - n + 1, dtype=np.int64)
    for j in range(n):
        keys = keys * vocab + ids[j : len(ids) - n + 1 + j]
    return keys


def count_pairs(ctx: np.ndarray, tok: np.ndarray, vocab: int):
    """Distinct (context, token) pairs, sorted, with how often each occurs."""
    contexts, rank = np.unique(ctx, return_inverse=True)
    keys, counts = np.unique(rank * vocab + tok, return_counts=True)
    return contexts[keys // vocab], keys % vocab, counts


def ngrams(ids: np.ndarray, n: int, vocab: int):
    return count_pairs(gram_keys(ids[:-1], n - 1, vocab), ids[n - 1 :], vocab)
```

Interpolated Kneser–Ney: continuation counts below the top order, and discounting and interpolation at every order above the first — code/u2/ngram.py

```python
class KneserNey:
    def __init__(self, ids: np.ndarray, order: int, vocab: int, discount: float = 0.75):
        self.order, self.vocab, self.d = order, vocab, discount
        self.tables = [None]
        for n in range(1, order + 1):
            if n == order:
                ctx, tok, counts = ngrams(ids, n, vocab)
            else:
```

```
                ctx, tok, _ = ngrams(ids, n + 1, vocab)
                ctx, tok, counts = count_pairs(ctx % vocab ** (n - 1), tok, vocab)
            self.tables.append(Table(ctx, tok, counts, vocab))

    def prob(self, history: np.ndarray, token: np.ndarray, n: int) -> np.ndarray:
        """P(token | the last n - 1 tokens of history), for arrays of positions."""
        t = self.tables[n]
        ctx = np.zeros(len(token), dtype=np.int64)
        for j in range(n - 1):
            ctx = ctx * self.vocab + history[:, history.shape[1] - n + 1 + j]
        count = t.lookup(ctx, token)
        if n == 1:
            return (count + 1.0) / (t.counts.sum() + self.vocab)
        total, distinct = t.context(ctx)
        lower = self.prob(history, token, n - 1)
        safe = np.maximum(total, 1)
        mixed = np.maximum(count - self.d, 0) / safe + self.d * distinct / safe * lower
        return np.where(total > 0, mixed, lower)
```

Each order's table holds the sorted contexts, with each one's total count and number of distinct continuations, and the sorted pairs with their counts. Looking up a whole validation set is two vectorised binary searches, one for the context and one for the token within it. The program builds a separate model of each order on the first 100 million training tokens and scores half a million validation tokens:

Scoring the validation set with every order — code/u2/ngram.py

```
k = args.order - 1
history = np.lib.stride_tricks.sliding_window_view(valid[:-1], k)
targets = valid[k:]
rows = []
for n in range(1, args.order + 1):
    start = time.perf_counter()
    model = KneserNey(train, n, tok.vocab_size)
    nats = float(-np.log(model.prob(history, targets, n)).mean())
    rows.append(
        {
            "order": n,
            "nats": nats,
            "bpb": nats / math.log(2) / bytes_per_token,
            "perplexity": math.exp(nats),
        }
    )
    print(
        f"{n}-gram: {nats:.3f} nats per token, {rows[-1]['bpb']:.3f} bits per byte, "
        f"perplexity {rows[-1]['perplexity']:7.1f}  ({time.perf_counter() - start:.0f} s)"
    )
```

N-gram models on TinyStories

```
$ uv run -m u2.ngram
1-gram: 5.829 nats per token, 2.024 bits per byte, perplexity   340.1  (3 s)
2-gram: 3.557 nats per token, 1.235 bits per byte, perplexity    35.1  (19 s)
3-gram: 2.851 nats per token, 0.990 bits per byte, perplexity    17.3  (35 s)
4-gram: 2.571 nats per token, 0.893 bits per byte, perplexity    13.1  (54 s)
5-gram: 2.474 nats per token, 0.859 bits per byte, perplexity    11.9  (75 s)
a sample from the 5-gram model:
  One day Tom was out in the park with her mom. They played on the swings and slide. Billy
    saw a muffin on the table. It was a different hotel was dark. Then they went home and
    told their friends about their fun adventure.One day, a little boy named Tim. He loved
    to run all day long.Once upon a time, in a big garden, there
```

Each extra token of context helps, with fast-diminishing returns: the unigram model's perplexity of 340 falls to 35 with one token of context and 17 with two, then only to 12 with four. Longer contexts are seen too rarely in 100 million tokens to estimate. The sample is typical of n-gram text. Each run of a few words reads naturally, because it was copied from real stories in pieces, but Tom becomes Billy, a muffin leads to a dark hotel, and three stories begin in eighty tokens without any of them being told.

PITFALL

Compare models only on the same text, with the same tokeniser or in bits per byte, and with every token counted. Skipping the hardest tokens, such as the first few of each document, or evaluating on text that overlaps the training data, makes any model look better than it is.

A neural language model

N-grams treat every context as unrelated to every other: having seen “the little dog ran” tells them nothing about “the little cat ran”. Bengio and colleagues’ neural language model of 2003 fixed this by giving every token a learned vector, an *embedding*, and predicting the next token from the concatenated embeddings of the context with a neural network. Tokens that behave alike get similar embeddings, so what is learned about one context transfers to similar ones.

A neural language model with a context of eight tokens — code/u2/mlp_lm.py

```
class MLPLanguageModel(nn.Module):
    def __init__(self, vocab: int, context: int, d_embed: int = 128, hidden: int = 1024):
        super().__init__()
        self.embed = nn.Embedding(vocab, d_embed)
        self.net = nn.Sequential(
            nn.Linear(context * d_embed, hidden),
            nn.LayerNorm(hidden),
            nn.ReLU(),
            nn.Linear(hidden, hidden),
            nn.LayerNorm(hidden),
            nn.ReLU(),
            nn.Linear(hidden, vocab),
        )

    def forward(self, ctx: torch.Tensor) -> torch.Tensor:
        return self.net(self.embed(ctx).flatten(1))
```

Training on random windows of the token file — code/u2/mlp_lm.py

```
start = time.perf_counter()
for step in range(1, args.steps + 1):
    for group in opt.param_groups:
        group["lr"] = 2e-3 * 0.5 * (1 + math.cos(math.pi * step / args.steps))
    rows = torch.from_numpy(
        windows(train, rng.integers(0, len(train) - k - 1, args.batch), k)
    ).to(dev)
    loss = F.cross_entropy(model(rows[:, :k]), rows[:, k])
    opt.zero_grad(set_to_none=True)
    loss.backward()
    opt.step()
    if step % 2000 == 0:
        print(
            f"step {step:5d}  training loss {loss.item():.3f}  "
            f"{time.perf_counter() - start:4.0f} s",
            flush=True,
        )
```

The neural model on the same validation tokens as the n-grams

```
$ uv run -m u2.mlp_lm
11.5M parameters, context of 8 tokens
step  2000  training loss 2.734    76 s
step  4000  training loss 2.575   151 s
step  6000  training loss 2.506   226 s
step  8000  training loss 2.384   302 s
step 10000  training loss 2.408   377 s
step 12000  training loss 2.374   456 s
step 14000  training loss 2.326   531 s
validation: 2.366 nats per token, 0.821 bits per byte, perplexity 10.7
```

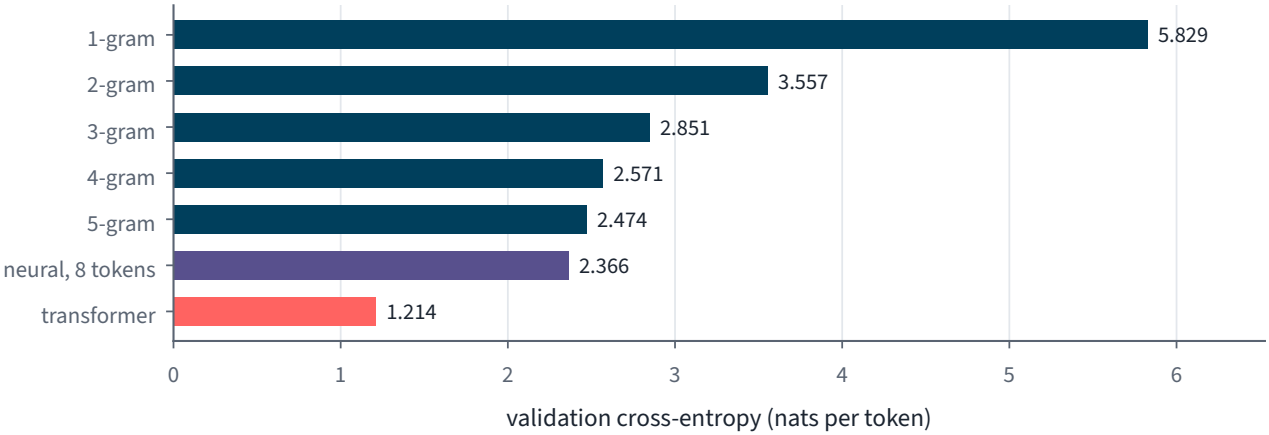


Figure 2.2.1. Validation cross-entropy of every model in this chapter, with Chapter 2.5's transformer, measured on its own validation windows, for comparison. Lower is better.

With a context of eight tokens, the neural model beats the 5-gram model, even though the 5-gram model has exact counts over 100 million tokens. It is limited in turn by its fixed window: it cannot use anything more than eight tokens back, and a story's characters, setting and plot are established far earlier. Chapter 2.3's attention lets every position look at every earlier one, and Chapter 2.5's transformer, trained on the same data, roughly halves the n-grams' loss (Figure 2.2.1).

KEY IDEA

A language model estimates the next token's distribution given the previous ones, and is judged by its cross-entropy on held-out text. Counted models fail because most contexts never occur in training. Learned embeddings let similar contexts share evidence, which is why a neural model does better.

IN PYTORCH

`torch.nn.Embedding(vocab, d)` is a lookup table: indexing a $V \times d$ matrix by token ids, which is the same as multiplying a one-hot vector by it, without forming the vector. Its gradient touches only the rows that were looked up. The cross-entropy over a vocabulary of 8,192 is `F.cross_entropy(logits, targets)`, as in Chapter 1.3.

Reading

- Daniel Jurafsky and James Martin, *Speech and Language Processing*, third edition draft, chapter 3: n-gram language models, perplexity and smoothing.
- Stanley Chen and Joshua Goodman, "An Empirical Study of Smoothing Techniques for Language Modeling", *Computer Speech and Language* 13(4), 1999.
- Yoshua Bengio, Réjean Ducharme, Pascal Vincent and Christian Jauvin, "A Neural Probabilistic Language Model", *JMLR* 3, 2003.
- Claude Shannon, "Prediction and Entropy of Printed English", *Bell System Technical Journal* 30(1), 1951.

Labs

LAB 2.2.1 Perplexity by hand — 1 hour

Goal. Compute a language model's measures by hand before relying on a program's.

Code. `code/u2/ngram.py`

1. On the corpus “the cat sat. the cat ran. the dog sat.”, treating words and full stops as tokens, write down the bigram counts and the maximum-likelihood bigram model.
2. Compute the probability, cross-entropy and perplexity of “the dog ran.” under it, and explain the result.
3. Repeat with add-one smoothing.

DONE WHEN

- [ ] Your two perplexities are correct and you can explain why the first is infinite.

EVIDENCE FOR THE NOTEBOOK

- The working, on paper or in the notebook.

LAB 2.2.2 Smoothing compared — 1.5 hours

Goal. Measure how much Kneser–Ney improves on simpler smoothing.

Code. `code/u2/ngram.py`

1. Add add-one smoothing and plain absolute discounting with raw lower-order counts to the program, as alternatives to Kneser–Ney.
2. Compare all three on bigrams, trigrams and 4-grams on the validation tokens.

DONE WHEN

- [ ] Your table has nine cross-entropies.
- [ ] You can explain, with an example from TinyStories, a context in which Kneser–Ney’s lower order helps.

EVIDENCE FOR THE NOTEBOOK

- The table and the example.

LAB 2.2.3 Data and order — 1 hour

Goal. See which models are limited by data.

Code. `code/u2/ngram.py`

1. Run the program with `--train-tokens` of 3, 10, 30, 100 and 300 million.
2. Plot cross-entropy against training tokens for each order.

DONE WHEN

- [ ] Your plot has five curves.
- [ ] You can say which orders are still improving at 300 million tokens and explain why.

EVIDENCE FOR THE NOTEBOOK

- The plot and the explanation.

LAB 2.2.4 Context length — 1.5 hours

Goal. Measure what more context gives the neural model.

Code. `code/u2/mlp_lm.py`

1. Train the neural model with contexts of 2, 4, 8 and 16 tokens, keeping everything else fixed.
2. Plot validation loss against context length, alongside the n-gram results of the same orders.

DONE WHEN

- [ ] You have four neural results and can compare each with the n-gram of the same context.
- [ ] You can explain why the neural model's advantage grows with context.

EVIDENCE FOR THE NOTEBOOK

- The plot and a paragraph.

Problem set

1. Show that perplexity is the geometric mean of the inverse probabilities the model assigns to the tokens of the test text.
2. A model assigns probability 0.25 to every token. What are its cross-entropy in nats and bits, its perplexity, and its bits per byte at 4 bytes per token?
3. How many distinct 5-grams could a vocabulary of 8,192 tokens form? Why is that number irrelevant to the memory a counted 5-gram model needs?
4. In absolute discounting, show that the probabilities of all tokens after a context sum to one when the lower-order model's do.
5. The neural model has 11.5 million parameters, most of them in its output layer. Which part would you shrink first to make it smaller without hurting it much, and why?

COMPLETION CHECKLIST

- [ ] I can state the chain rule of probability and what an n-gram model assumes.
- [ ] I can compute cross-entropy, perplexity and bits per byte, and convert between them.
- [ ] I can explain smoothing, absolute discounting and Kneser–Ney's continuation counts.
- [ ] I can explain why a neural language model generalises across contexts where counts cannot.
- [ ] Lab 2.2.1–2.2.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.3

Attention

TIME	6 hours
YOU BUILD	Scaled dot-product attention derived as a soft dictionary lookup and written in NumPy, with causal masking and multiple heads, matched to PyTorch's fused kernel; attention assembled from `tiny`'s operations, its gradients matched too; the reason for the $1/\sqrt{d}$ scale, measured; and a small transformer that learns to copy, with its attention drawn.
NEEDS	Chapters 1.2 and 2.2.

WHY IT MATTERS

Chapter 2.2's neural model could see eight tokens back and no further, so a story's heroine, introduced in the first sentence, was invisible to it by the third. *Attention* removes the window. It lets every position gather information from every earlier position, with weights computed from what those positions contain, so a model can look up the name it needs wherever it was mentioned. It is the central operation of the transformer, and so of every modern language model, and this chapter builds it from its definition.

SKIP TEST

1. Write attention for queries Q, keys K and values V, giving the shape of each tensor and of the result for a sequence of length T.
2. Why are the attention scores divided by $\sqrt{d}$?
3. What does a causal mask do, and how is it applied before the softmax?
4. What can eight heads of size 64 do that one head of size 512 cannot?

If all four are easy, read the sections on scale and on learned attention, and do Labs 2.3.2 and 2.3.3.

Core ideas

A soft dictionary

A dictionary maps keys to values: given a query, find the key that equals it and return its value. Attention makes every part of that soft. Each position j of a sequence has a *key* vector k_j and a *value* vector v_j. A position i forms a *query* q_i, scores every key by its dot product with the query, turns the scores into weights with a softmax, and receives the weighted average of the values:

$$\text{out}_i = \sum_j a_{ij} v_j, \quad a_{ij} = \frac{\exp\left(q_i \cdot k_j / \sqrt{d}\right)}{\sum_{j'} \exp\left(q_i \cdot k_{j'} / \sqrt{d}\right)}.$$

A query that closely matches one key retrieves almost only that key's value, and one that matches several retrieves a blend. Because every step is differentiable, the model can learn what to put in queries, keys and values. Stacking the vectors as rows of matrices gives the whole computation in three matrix operations:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V,$$

with Q of shape $T \times d$, K and V of shape $T \times d$, the score matrix $T \times T$, and the result $T \times d$. In *self-attention* the queries, keys and values are all computed from the same sequence by three learned linear maps, $Q = XW_Q$, $K = XW_K$, $V = XW_V$.

Causal masking

A language model predicts each token from the tokens before it, so position i must not see positions after it, or training would teach it to copy the answer. A *causal mask* sets the scores above the diagonal to $-\infty$ before the softmax, making their weights exactly zero:

Attention in NumPy, with an optional causal mask — code/u2/attention.py

```
def softmax(z: np.ndarray) -> np.ndarray:
    e = np.exp(z - z.max(axis=-1, keepdims=True))
    return e / e.sum(axis=-1, keepdims=True)


def attention(q: np.ndarray, k: np.ndarray, v: np.ndarray, causal: bool = False) -> np.ndarray:
    scores = q @ k.swapaxes(-1, -2) / np.sqrt(q.shape[-1])
    if causal:
        T = q.shape[-2]
        scores = np.where(np.tril(np.ones((T, T), dtype=bool)), scores, -np.inf)
    return softmax(scores) @ v
```

All positions are still computed at once: one pass over a sequence of T tokens produces T predictions, each using only its own past, which is what makes training on whole sequences efficient.

Multiple heads

A single attention computes one set of weights per position. *Multi-head attention* splits the d dimensions into H heads of d/H dimensions each, runs attention independently in each, and concatenates the results before a final linear map, W_O. Each head can attend to something different: one to the previous token, one to the subject of the sentence, one to the matching quotation mark. The cost is the same as one head of the full width.

Multi-head causal self-attention for one sequence — code/u2/attention.py

```
def multi_head(x: np.ndarray, wq, wk, wv, wo, heads: int, causal: bool = True) -> np.ndarray:
    T, d = x.shape
    split = lambda m: m.reshape(T, heads, d // heads).transpose(1, 0, 2)
    q, k, v = split(x @ wq), split(x @ wk), split(x @ wv)
    y = attention(q, k, v, causal)
    return y.transpose(1, 0, 2).reshape(T, d) @ wo
```

PyTorch provides attention as one fused operation, `scaled_dot_product_attention`, which this course's transformer uses:

The same computation with PyTorch's kernel — code/u2/attention.py

```
tq, tk, tv = (
    torch.tensor(x @ w).view(T, heads, d // heads).transpose(0, 1) for w in (wq, wk, wv)
)
y = F.scaled_dot_product_attention(tq, tk, tv, is_causal=True)
ref = (y.transpose(0, 1).reshape(T, d) @ torch.tensor(wo)).numpy()
```

NumPy against PyTorch, and `tiny`'s gradients against PyTorch's

```
$ uv run -m u2.attention
multi-head causal attention in NumPy against PyTorch: largest difference 4.4e-16
attention built from tiny's operations: gradients match PyTorch's to 4.4e-16
```

The second line checks the gradients. Attention is built from operations `tiny` already has (matrix products, a transpose, a softmax written as the exponential of `log_softmax`, and an additive mask), so Chapter 1.2's engine differentiates it without any new rule, and its gradients agree with PyTorch's to within float64 rounding error.

Why divide by the square root of the dimension

If the entries of q and k are independent with unit variance, their dot product is a sum of d terms of unit variance, with standard deviation $\sqrt{d}$. For $d = 64$ the scores spread over a range of about ± 16, and a softmax of numbers that far apart puts nearly all the weight on the largest. The softmax is then *saturated*: the gradient through it is nearly zero, and the model cannot learn which keys to prefer. Dividing by $\sqrt{d}$ restores unit variance whatever the dimension:

The largest attention weight among 128 keys, with and without the scale — code/u2/attention.py

```
for dim in (16, 64, 256):
    keys = rng.normal(size=(1000, 128, dim))
    query = rng.normal(size=dim)
    scores = keys @ query
    unscaled = softmax(scores).max(1).mean()
    scaled = softmax(scores / np.sqrt(dim)).max(1).mean()
    print(
        f"d = {dim:3d}: scores have standard deviation {scores.std():4.1f}; "
        f"the largest of 128 weights "
        f"averages {unscaled:.2f} unscaled, {scaled:.2f} scaled"
    )
```

Unscaled attention saturates as the dimension grows

```
d =  16: scores have standard deviation  3.6; the largest of 128 weights averages 0.50
    unscaled, 0.06 scaled
d =  64: scores have standard deviation  7.9; the largest of 128 weights averages 0.76
    unscaled, 0.07 scaled
d = 256: scores have standard deviation 14.8; the largest of 128 weights averages 0.87
    unscaled, 0.06 scaled
```

What attention costs

The score matrix has T^2 entries per head. Computing it takes $2T^2d$ FLOPs per head in total, and the weighted sum as many again, so attention's cost grows with the square of the sequence length, while everything else in a transformer grows linearly. At the 512 tokens of this unit's models the quadratic term is small, and at 100,000 tokens it dominates. Storing the matrix for the backward pass also costs memory quadratic in T, which Chapter 4.4's FlashAttention avoids by never storing it.

Attention also takes no account of order. Permuting the keys and values permutes the weights to match and leaves the output unchanged, so attention itself carries no information that position 3 comes before position 4. Chapter 2.4 adds that information with rotary position embeddings.

Attention, learned

To see what attention learns, the program trains a small transformer, Chapter 2.4's model shrunk to 64 dimensions, on a task that needs it: a sequence of 16 random tokens followed by the same 16 again. The first half is unpredictable, and each token of the second half can be predicted exactly. At a position in the second half, the next token is the one that followed the current token's earlier occurrence, 16 positions back, and so sits 15 positions back. Attending there and copying the token found solves the task.

Random tokens, repeated; only the repeat is scored — code/u2/copy_task.py

```
def batch(size: int, half: int, vocab: int, dev) -> tuple[torch.Tensor, torch.Tensor]:
    first = torch.randint(0, vocab, (size, half), device=dev)
    seq = torch.cat([first, first], dim=1)
    targets = seq[:, 1:].clone()
    targets[:, : half - 1] = -100
    return seq[:, :-1], targets
```

A two-layer transformer trained on the task code/u2/copy_task.py

```
config = gpt.Config(
    vocab_size=vocab,
    d_model=64,
    n_layers=args.layers,
    n_heads=2,
    n_kv_heads=2,
    d_ff=128,
    max_seq=2 * half,
)
model = gpt.GPT(config).to(dev)
opt = torch.optim.AdamW(model.parameters(), lr=3e-3)
for step in range(1, args.steps + 1):
    x, y = batch(64, half, vocab, dev)
    _, loss = model(x, y)
    opt.zero_grad()
    loss.backward()
    opt.step()
    if step % 500 == 0:
        print(f"step {step:4d}  loss on the repeated half {loss.item():.3f}")
```

The model learns to copy, and its last layer attends 15 positions back

```
$ uv run -m u2.copy_task
step  500  loss on the repeated half 0.002
step 1000  loss on the repeated half 0.001
step 1500  loss on the repeated half 0.000
step 2000  loss on the repeated half 0.000
step 2500  loss on the repeated half 0.000
step 3000  loss on the repeated half 0.000
copying accuracy on the repeated half: 100.00% (chance is 1.6%)
head 0: in the repeated half, the strongest attention is 15 positions back for 16 of 16
    positions, with mean weight 0.98
head 1: in the repeated half, the strongest attention is 15 positions back for 16 of 16
    positions, with mean weight 0.98
```

The trained model copies the repeated half without error. In the second half, both heads of its last layer put 98% of their weight on the position 15 tokens back (Figure 2.3.1). A one-layer model trained the same

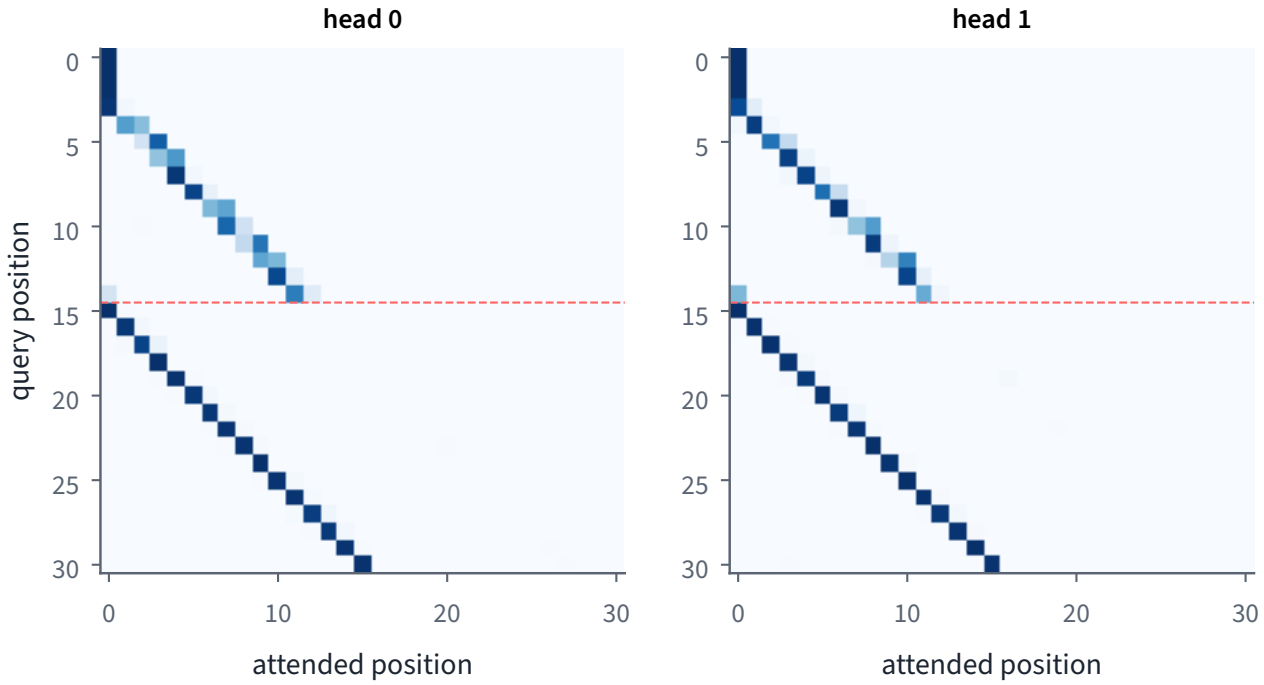


Figure 2.3.1. Attention weights of the last layer's two heads for one sequence. In the second half, each position attends to the position 15 tokens earlier, which appears as the bright diagonal.

way gets about three quarters right. Lab 2.3.2 asks you to find out why, and Chapter 2.8 finds the same two-layer mechanism, the *induction head*, inside the model that Chapter 2.5 trains on stories.

KEY IDEA

Attention is a differentiable lookup: queries are scored against keys, the scores become weights by a softmax, and the result is a weighted average of values. The causal mask restricts each position to its past, as a language model requires. The $1/\sqrt{d}$ scale keeps the softmax from saturating, and the cost grows with the square of the sequence length.

IN PYTORCH

`F.scaled_dot_product_attention(q, k, v, is_causal=True)` computes this chapter's attention for tensors of shape (batch, heads, length, head size) and picks the fastest kernel for the device: FlashAttention on recent NVIDIA GPUs, and a Metal kernel on MPS. Its `attn_mask` takes either a boolean mask, `True` where attention is allowed, or an additive one. `torch.nn.MultiheadAttention` wraps the projections as well, with a different layout, and this course writes its own.

Reading

- Dzmitry Bahdanau, Kyunghyun Cho and Yoshua Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate", ICLR 2015: attention's origin.
- Ashish Vaswani and others, "Attention Is All You Need", NeurIPS 2017, section 3.2.
- Nelson Elhage and others, "A Mathematical Framework for Transformer Circuits", Anthropic, 2021: attention heads as readers and writers of a residual stream.

Labs

LAB 2.3.1 Attention by hand — 1 hour

Goal. Compute attention on paper for a sequence of three positions.

Code. `code/u2/attention.py`

1. Choose three positions with two-dimensional queries, keys and values of your own. Compute the causal attention output by hand, without the scale.
2. Check your numbers with `attention()`, after removing its scale or multiplying your queries by $\sqrt{2}$.

DONE WHEN

- [ ] Your hand computation matches the program to three decimal places.

EVIDENCE FOR THE NOTEBOOK

- The working and the program's output.

LAB 2.3.2 One layer against two — 2 hours

Goal. Find out why a one-layer model copies poorly.

Code. `code/u2/copy_task.py`

1. Train the copy task with `--layers 1` and `--layers 2`, three seeds each with `--seed`, and record the accuracies.
2. Draw the attention of every head of every layer for one sequence in each case.
3. Describe what the first layer of the two-layer model does, and why the one-layer model cannot easily do the same.

DONE WHEN

- [ ] Your table has six accuracies and your figures show every head.
- [ ] Your explanation names what information the second layer needs and where it comes from.

EVIDENCE FOR THE NOTEBOOK

- The table, the figures and the explanation.

LAB 2.3.3 Without the scale — 1 hour

Goal. Measure how saturation slows learning.

Code. `code/u2/copy_task.py`

1. Patch the model's attention to omit the $1/\sqrt{d}$ scale; the simplest way is to multiply the queries by $\sqrt{d}$ before calling the kernel.
2. Train the copy task with and without the patch and plot the loss curves, with heads of 32 and of 128 dimensions.

DONE WHEN

- [ ] You have four loss curves.
- [ ] You can say how much the missing scale slowed learning at each head size, and why the effect grows with the size.

EVIDENCE FOR THE NOTEBOOK

- The plot and a paragraph.

LAB 2.3.4 The quadratic cost — 1 hour

Goal. Measure attention's time and memory against sequence length.

Code. `code/u2/attention.py`

1. Time materialised causal attention, as in `attention()` but in PyTorch on your device, for 8 heads of 64 dimensions and lengths from 256 to 8,192, and record the peak memory.
2. Time `scaled_dot_product_attention` at the same lengths.
3. Fit the times to aT^2 and say at what length attention would take as long as the rest of a layer of Chapter 2.5's model.

DONE WHEN

- [ ] Your table and plot cover at least six lengths for both implementations.
- [ ] Your fitted constant predicts the 8,192 measurement within 20%.

EVIDENCE FOR THE NOTEBOOK

- The table, the plot and the prediction.

Problem set

1. Show that permuting the keys and values together, with the same permutation, leaves the output of non-causal attention unchanged.
2. Count the FLOPs of causal multi-head attention for $T = 512$, $d = 512$ and 8 heads, separately for the projections and for the scores and weighted sum.
3. If every key is identical, what does attention output, whatever the queries? What if every query is zero?
4. Why does setting masked scores to $-\infty$ give exactly zero weight, while setting them to -10^9 gives nearly zero? When could the difference matter?

5. A head always attends to the previous position. Write down queries and keys, using a position-dependent component, that would produce that pattern.

COMPLETION CHECKLIST

- [ ] I can derive attention as a soft lookup and give the shape of every tensor in it.
- [ ] I can implement causal multi-head attention and check it against PyTorch.
- [ ] I can explain the $1/\sqrt{d}$ scale and the quadratic cost.
- [ ] I can read an attention pattern and say what a head is doing.
- [ ] Lab 2.3.1–2.3.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.4

The transformer

TIME	6 hours
YOU BUILD	A decoder-only transformer in about 200 lines, with pre-norm blocks, RMS normalisation, rotary position embeddings, grouped-query attention and a SwiGLU feed-forward network, written so that it can load the weights of the Llama family; its parameters counted by formula and checked; its FLOPs per token derived; and the three tests that every transformer should pass.
NEEDS	Chapters 1.7 and 2.3.

WHY IT MATTERS

The transformer is the architecture of nearly every language model in use, from this course's 28 million parameters to models ten thousand times larger, and the differences between them are mostly a handful of well-understood design choices. This chapter assembles one from the pieces of earlier chapters, explains each choice, and checks the result. The details matter because Unit 3 loads SmolLM2's published weights into this code, and every choice here must match its designers' or the model will produce nonsense.

SKIP TEST

1. Draw a pre-norm transformer block, and say where the residual connections and the normalisations go.
2. Rotary position embeddings rotate queries and keys by an angle that depends on position. Why does the dot product of a rotated query and key depend only on their distance?
3. What does grouped-query attention save, and what does it cost?
4. How many parameters does one block have with $d = 512$, a feed-forward width of 1,408, 8 query heads and 4 key-value heads of 64 dimensions?

If all four are easy, read the sections on rotary embeddings and on counting, and do Labs 2.4.3 and 2.4.4.

Core ideas

The shape of a decoder-only transformer

A token sequence enters as embedding vectors, one row of a learned table per token. A stack of identical *blocks* then transforms them, each adding to the vectors what it computes, so that the vector at each position, the *residual stream*, accumulates information as it rises through the stack. Each block does two things in turn: attention, which moves information between positions, and a feed-forward network, which transforms each position on its own. A final normalisation and a linear map to the vocabulary turn the top of the residual stream into logits for the next token (Figure 2.4.1).

All the dimensions are set in one configuration:

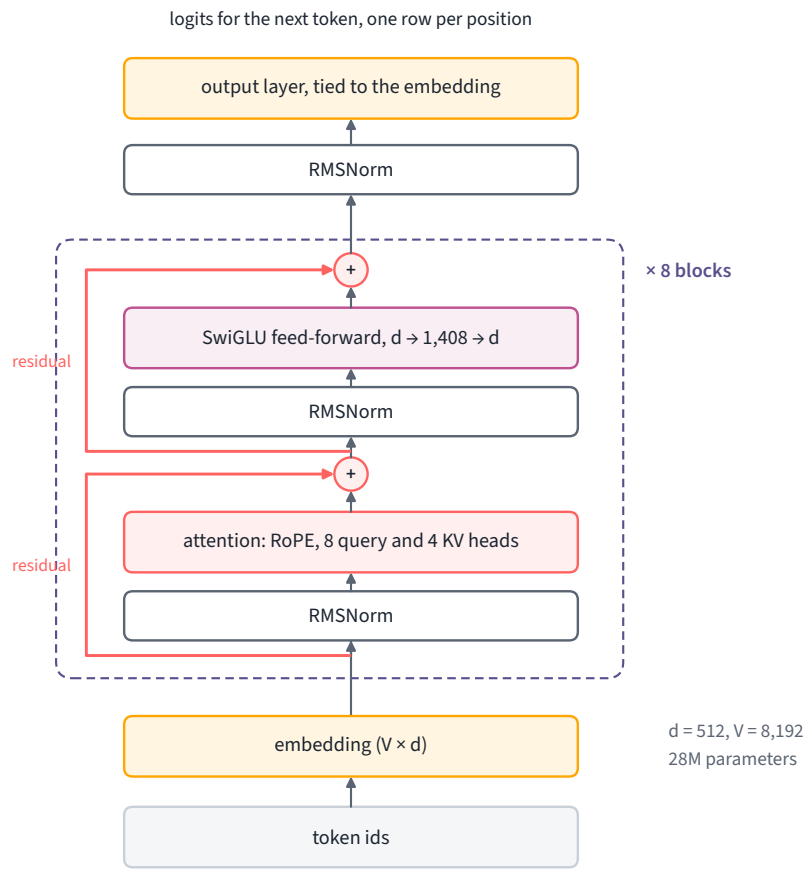


Figure 2.4.1. The decoder-only transformer of this course. Each of the eight blocks adds the output of attention and of the feed-forward network to the residual stream. The output layer shares its weights with the embedding.

The configuration, with the defaults of Chapter 2.5's model — code/aif/gpt.py

```
@dataclass
class Config:
    vocab_size: int = 8192
    d_model: int = 512
    n_layers: int = 8
    n_heads: int = 8
    n_kv_heads: int = 4
    d_ff: int = 1408
    max_seq: int = 512
    rope_theta: float = 10000.0
    norm_eps: float = 1e-5
    tie_embeddings: bool = True

    @property
    def head_dim(self) -> int:
        return self.d_model // self.n_heads
```

Normalisation and blocks

Chapter 1.7 showed why deep networks need normalisation and residual connections, and why pre-norm trains more easily than post-norm. The transformer uses RMS normalisation before each half of each block, computed in float32 whatever the model's precision, since a sum of squares overflows half precision easily:

RMS normalisation code/aif/gpt.py

```
class RMSNorm(nn.Module):
    def __init__(self, dim: int, eps: float):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        dtype = x.dtype
        x = x.float()
        x = x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        return self.weight * x.to(dtype)
```

A pre-norm block: two residual additions, each of a normalised sublayer code/aif/gpt.py

```
class Block(nn.Module):
    def __init__(self, c: Config):
        super().__init__()
        self.attn_norm = RMSNorm(c.d_model, c.norm_eps)
        self.attn = Attention(c)
        self.mlp_norm = RMSNorm(c.d_model, c.norm_eps)
        self.mlp = SwiGLU(c)

    def forward(self, x, cos, sin, cache=None, layer=0):
        x = x + self.attn(self.attn_norm(x), cos, sin, cache, layer)
        return x + self.mlp(self.mlp_norm(x))
```

Rotary position embeddings

Attention takes no account of order, so the model must be given each token's position. The original transformer added a vector depending on the position to each embedding. Modern models instead *rotate* the queries and keys. Split each head's vector into pairs of dimensions; rotate pair k by the angle $p\theta_k$, where p is the token's position and the frequencies $\theta_k = 10000^{-2k/h_d}$, with h_d the head size, range from one radian per position down to a tiny fraction of one.

DERIVATION

Treat one pair of dimensions as a complex number, rotated with frequency θ. The query at position m becomes $qe^{im\theta}$ and the key at position n becomes $ke^{in\theta}$, with i the imaginary unit. The pair's contribution to the dot product is the real part of the rotated query times the conjugate of the rotated key,

$$\mathrm{Re}\left(qe^{im\theta}\cdot\overline{ke^{in\theta}}\right)=\mathrm{Re}\left(q\bar{k}e^{i(m-n)\theta}\right),$$

which depends on the positions only through $m-n$. Summing over the pairs, the score between a query and a key depends only on their content and their relative position.

Relative position is what attention needs, because "the previous token" or "the token after the last occurrence of this one" means the same thing anywhere in the sequence. The implementation follows the convention of Hugging Face's Llama code, which rotates the first half of a head's dimensions against the second half rather than adjacent pairs. The two are equivalent up to a fixed permutation of the dimensions, but loading another model's weights requires the same convention.

The rotation tables and their application — code/aif/gpt.py

```python
def rope_tables(head_dim: int, length: int, theta: float) -> tuple[torch.Tensor, torch.Tensor]:
    inv_freq = 1.0 / theta ** (torch.arange(0, head_dim, 2).float() / head_dim)
    angles = torch.outer(torch.arange(length).float(), inv_freq)
    angles = torch.cat([angles, angles], dim=-1)
    return angles.cos(), angles.sin()


def rotate_half(x: torch.Tensor) -> torch.Tensor:
    x1, x2 = x.chunk(2, dim=-1)
    return torch.cat([-x2, x1], dim=-1)


def apply_rope(x: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor) -> torch.Tensor:
    return x * cos + rotate_half(x) * sin
```

Grouped-query attention

During generation, every layer keeps the keys and values of all previous tokens, Chapter 2.7's KV cache, and for long contexts that cache dominates memory. *Grouped-query attention* lets several query heads share one key-value head: with 8 query heads and 4 key-value heads, each pair of query heads reads the same keys and values. The cache and the key and value projections halve, and the quality cost is small. SmolLM2-135M shares each of 3 key-value heads among 3 query heads.

Attention with rotary embeddings, grouped key-value heads and an optional KV cache — code/aif/gpt.py

```python
class Attention(nn.Module):
    def __init__(self, c: Config):
        super().__init__()
        self.n_heads, self.n_kv_heads, self.head_dim = c.n_heads, c.n_kv_heads, c.head_dim
        self.q = nn.Linear(c.d_model, c.n_heads * c.head_dim, bias=False)
        self.k = nn.Linear(c.d_model, c.n_kv_heads * c.head_dim, bias=False)
        self.v = nn.Linear(c.d_model, c.n_kv_heads * c.head_dim, bias=False)
        self.o = nn.Linear(c.n_heads * c.head_dim, c.d_model, bias=False)

    def forward(self, x, cos, sin, cache: KVCache | None = None, layer: int = 0):
        B, T, _ = x.shape
        q = self.q(x).view(B, T, self.n_heads, self.head_dim).transpose(1, 2)
        k = self.k(x).view(B, T, self.n_kv_heads, self.head_dim).transpose(1, 2)
        v = self.v(x).view(B, T, self.n_kv_heads, self.head_dim).transpose(1, 2)
        q, k = apply_rope(q, cos, sin), apply_rope(k, cos, sin)
        past = 0
        if cache is not None:
            past = cache.length
            k, v = cache.update(layer, k, v)
        group = self.n_heads // self.n_kv_heads
        k, v = k.repeat_interleave(group, dim=1), v.repeat_interleave(group, dim=1)
        if past == 0:
            y = F.scaled_dot_product_attention(q, k, v, is_causal=True)
        else:
            mask = torch.ones(T, past + T, dtype=torch.bool, device=x.device).tril(
                diagonal=past
            )
            y = F.scaled_dot_product_attention(q, k, v, attn_mask=mask)
        return self.o(y.transpose(1, 2).reshape(B, T, -1))
```

The feed-forward network

Each position then passes through the same two-layer network. Modern models use a gated variant, *SwiGLU*: one projection, passed through the smooth SiLU nonlinearity $x\sigma(x)$, multiplies another element-wise before the down projection. Gating lets the network switch features on and off depending on the input, and consistently trains slightly better than a plain MLP with the same number of parameters. Since it has three matrices instead of two, its width is conventionally about $8/3$ of the model dimension, rounded up to a multiple of 64.

The SwiGLU feed-forward network code/aif/gpt.py

```
class SwiGLU(nn.Module):
    def __init__(self, c: Config):
        super().__init__()
        self.gate = nn.Linear(c.d_model, c.d_ff, bias=False)
        self.up = nn.Linear(c.d_model, c.d_ff, bias=False)
        self.down = nn.Linear(c.d_ff, c.d_model, bias=False)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.down(F.silu(self.gate(x)) * self.up(x))
```

The whole model

Embedding, blocks, final norm and a tied output layer, with the loss when targets are given code/aif/gpt.py

```
class GPT(nn.Module):
    def __init__(self, c: Config):
        super().__init__()
        self.config = c
        self.embed = nn.Embedding(c.vocab_size, c.d_model)
        self.blocks = nn.ModuleList(Block(c) for _ in range(c.n_layers))
        self.norm = RMSNorm(c.d_model, c.norm_eps)
        self.head = nn.Linear(c.d_model, c.vocab_size, bias=False)
        if c.tie_embeddings:
            self.head.weight = self.embed.weight
        cos, sin = rope_tables(c.head_dim, c.max_seq, c.rope_theta)
        self.register_buffer("cos", cos, persistent=False)
        self.register_buffer("sin", sin, persistent=False)
        self.apply(self._init)
        for block in self.blocks:
            for proj in (block.attn.o, block.mlp.down):
                nn.init.normal_(proj.weight, std=0.02 / math.sqrt(2 * c.n_layers))

    @staticmethod
    def _init(m: nn.Module) -> None:
        if isinstance(m, (nn.Linear, nn.Embedding)):
            nn.init.normal_(m.weight, std=0.02)

    def forward(
        self,
        idx: torch.Tensor,
        targets: torch.Tensor | None = None,
        cache: KVCache | None = None,
    ):
        T = idx.shape[1]
        start = cache.length if cache is not None else 0
        cos, sin = self.cos[start : start + T], self.sin[start : start + T]
        x = self.embed(idx)
        for i, block in enumerate(self.blocks):
            x = block(x, cos, sin, cache, i)
        if cache is not None:
            cache.length += T
        logits = self.head(self.norm(x))
        if targets is None:
            return logits
        loss = F.cross_entropy(
            logits.float().flatten(0, 1), targets.flatten(), ignore_index=-100
        )
        return logits, loss
```

The output layer *ties* its weights to the embedding: the same matrix maps tokens to vectors on the way in and vectors to logits on the way out. That saves $V \times d$ parameters, a seventh of this model, and works as well as separate matrices at this scale. Initialisation follows GPT-2: weights drawn with standard deviation 0.02, and the two projections that write into the residual stream scaled down by $\sqrt{2L}$, so that the sum of $2L$ contributions keeps a steady size.

Counting parameters and FLOPs

Per block, attention has $d \cdot h_d(2H + 2H_{\text{kv}})$ parameters, with h_d the head size: queries and output projections for H heads, keys and values for H_{kv}. The feed-forward network has $3d \cdot d_{\text{ff}}$, and the two norms $2d$. A forward pass multiplies each parameter by one input once per token, two FLOPs per parameter. The backward pass costs twice the forward pass, as Chapter 1.2 derived, so training costs about six FLOPs per parameter per token. Attention's scores and weighted sums add $6LTd$ per token for a context of T, and the output layer, which multiplies by the embedding matrix, counts with the parameters even though it is tied.

Parameters, and training FLOPs per token — code/aif/gpt.py

```
def param_count(c: Config, embedding: bool = True) -> int:
    attn = c.d_model * c.head_dim * (2 * c.n_heads + 2 * c.n_kv_heads)
    mlp = 3 * c.d_model * c.d_ff
    per_layer = attn + mlp + 2 * c.d_model
    total = c.n_layers * per_layer + c.d_model
    if embedding:
        total += c.vocab_size * c.d_model * (1 if c.tie_embeddings else 2)
    return total


def train_flops_per_token(c: Config, seq: int) -> float:
    matmul_params = param_count(c, embedding=False) + c.vocab_size * c.d_model
    return 6 * matmul_params + 6 * c.n_layers * seq * c.d_model
```

Where the parameters are — code/u2/gpt_check.py

```
def breakdown(c: gpt.Config) -> dict[str, int]:
    attn = c.d_model * c.head_dim * (2 * c.n_heads + 2 * c.n_kv_heads)
    return {
        "embedding (tied with the output)": c.vocab_size * c.d_model,
        "attention, all layers": c.n_layers * attn,
        "feed-forward, all layers": c.n_layers * 3 * c.d_model * c.d_ff,
        "norms": c.n_layers * 2 * c.d_model + c.d_model,
    }
```

This course's model and SmolLM2-135M, part by part

```
$ uv run -m u2.gpt_check
this course's TinyStories model: 8 layers, d_model 512, 8 query and 4 key-value heads, d_ff
    1408, vocabulary 8,192
  embedding (tied with the output)      4,194,304   15.1%
  attention, all layers                 6,291,456   22.6%
  feed-forward, all layers             17,301,504   62.2%
  norms                                     8,704    0.0%
  total                                27,795,968   179 MFLOP per training token at 512
    tokens of context
SmolLM2-135M: 30 layers, d_model 576, 9 query and 3 key-value heads, d_ff 1536, vocabulary
    49,152
  embedding (tied with the output)     28,311,552   21.0%
  attention, all layers                26,542,080   19.7%
  feed-forward, all layers             79,626,240   59.2%
  norms                                    35,136    0.0%
  total                               134,515,008   860 MFLOP per training token at 512
    tokens of context
```

Six-tenths of each model is its feed-forward networks. The embedding is a seventh of the course's model and a fifth of SmolLM2's, whose vocabulary is six times larger. Training the course's model costs 179 MFLOPs per token, so Chapter 2.5's 300 million tokens need 5.4×10^{16} FLOPs, just under five hours at the 3.2 TFLOP/s that the M2 Pro sustains on it.

Three tests every transformer should pass

A transformer can contain bugs that still let it train, to a worse result. Three cheap tests catch most of them.

Causality: changing later tokens must not change earlier outputs code/u2/gpt_check.py

```
x = torch.randint(0, ours.vocab_size, (1, 64))
changed = x.clone()
changed[0, 40:] = torch.randint(0, ours.vocab_size, (24,))
with torch.no_grad():
    a, b = model(x), model(changed)
before = (a[0, :40] - b[0, :40]).abs().max().item()
after = (a[0, 40:] - b[0, 40:]).abs().max().item()
```

Relative positions: the same offset must give the same score anywhere code/u2/gpt_check.py

```
q, k = torch.randn(64), torch.randn(64)
cos, sin = gpt.rope_tables(64, 512, 10000.0)
rotate = lambda v, p: gpt.apply_rope(v, cos[p], sin[p])
pairs = [(10, 3), (110, 103), (400, 393)]
dots = [float(rotate(q, i) @ rotate(k, j)) for i, j in pairs]
```

Capacity: a small model must memorise a single batch code/u2/gpt_check.py

```
small = gpt.GPT(
    gpt.Config(
        vocab_size=ours.vocab_size,
        d_model=128,
        n_layers=2,
        n_heads=2,
        n_kv_heads=1,
        d_ff=384,
        max_seq=64,
    )
)
x = torch.randint(0, ours.vocab_size, (4, 65))
opt = torch.optim.AdamW(small.parameters(), lr=3e-3)
for step in range(300):
    _, loss = small(x[:, :-1], x[:, 1:])
    opt.zero_grad()
    loss.backward()
    opt.step()
```

All three pass

```
changing tokens 40 to 63 moves the logits at 0 to 39 by 0.0e+00 and at 40 to 63 by 2.44
rotated q.k at positions (10, 3), (110, 103) and (400, 393): -4.4043, -4.4043, -4.4043
one batch of random tokens memorised: loss 0.0028 after 300 steps
```

A model that leaks the future scores impossibly well in training and generates nonsense. A wrong rotation convention trains but cannot load another model's weights. A model that cannot memorise four sequences of random tokens has a bug, whatever its loss on real text.

KEY IDEA

A modern transformer is a stack of pre-norm blocks, each adding attention and a gated feed-forward network to a residual stream, with positions supplied by rotating queries and keys. Training costs about six FLOPs per parameter per token, and most of the parameters are in the feed-forward networks.

IN PYTORCH

Hugging Face's `LlamaForCausalLM` is this chapter's model with different names: `q_proj` for `attn.q`, `input_layernorm` for `attn_norm`, `gate_proj` for `mlp.gate`, and so on, a mapping Chapter 3.1 uses to load weights. `torch.nn.RMSNorm` and `F.silu` are built in. `torch.compile(model)` fuses the small element-wise operations, the norms, the rotation, SiLU and the residual additions, into fewer kernels, which Chapter 2.5 measures and Chapter 4.2 explains.

Reading

- Ashish Vaswani and others, "Attention Is All You Need", NeurIPS 2017, section 3.
- Jianlin Su and others, "RoFormer: Enhanced Transformer with Rotary Position Embedding", 2021.
- Noam Shazeer, "GLU Variants Improve Transformer", 2020.
- Joshua Ainslie and others, "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints", EMNLP 2023.
- Hugo Touvron and others, "LLaMA: Open and Efficient Foundation Language Models", 2023, section 2.2: the architecture this chapter follows.

Labs

LAB 2.4.1 Tensor shapes — 1 hour

Goal. Know the shape of every tensor in the forward pass.

Code. `code/aif/gpt.py`

1. For a batch of 2 sequences of 16 tokens, write down the shape of every tensor in one block's forward pass, including inside attention.
2. Check your list with forward hooks that print the shapes of each module's inputs and outputs.

DONE WHEN

- [ ] Every shape in your list matches the hooks' output.

EVIDENCE FOR THE NOTEBOOK

- The annotated list.

LAB 2.4.2 Ablations at small scale — 2 hours

Goal. Measure what each modern design choice contributes.

Code. `code/u2/pretrain.py`

1. Make three variants of the model: LayerNorm instead of RMSNorm; learned absolute position embeddings instead of rotary; a GELU MLP with the same parameter count instead of SwiGLU.
2. Train each, and the unchanged model, with `--preset small --tokens 20e6`, and compare validation losses.

DONE WHEN

- [ ] You have four validation losses from runs with the same data and seed.
- [ ] You can say which choice mattered most at this scale, and whether the differences exceed the run-to-run variation, measured with a second seed.

EVIDENCE FOR THE NOTEBOOK

- The table of losses with both seeds.

LAB 2.4.3 Fewer key-value heads — 1.5 hours

Goal. Trade memory against quality with grouped-query attention.

Code. `code/u2/pretrain.py`

1. Compute the KV-cache size per token and the attention parameters for 4, 2 and 1 key-value heads with the `small` preset's 4 query heads.
2. Train the `small` preset with each, `--kv-heads 4` and so on, for 20 million tokens, and compare validation losses.

DONE WHEN
- [ ] Your table has the cache size, parameters and loss for all three settings.
- [ ] You can recommend a setting for a model whose cache must fit in a quarter of the memory.

EVIDENCE FOR THE NOTEBOOK
- The table and the recommendation.

LAB 2.4.4 Count SmolLM2 by hand — 1 hour

Goal. Derive a real model's size and cost from its configuration.

Code. `code/u2/gpt_check.py`

1. From SmolLM2-360M's `config.json`, compute its parameters by part and its training FLOPs per token at 2,048 tokens of context.
2. Check your parameter count by loading the model with `gpt.from_huggingface`.
3. Estimate how long the M2 Pro would take to train it on four trillion tokens, its published budget.

DONE WHEN
- [ ] Your parameter count is exact.
- [ ] Your time estimate is stated with the throughput it assumes.

EVIDENCE FOR THE NOTEBOOK
- The calculation and the check.

Problem set

1. Show that applying the same rotation to a query and a key leaves their dot product unchanged. Why does that make rotary embeddings relative?
2. With 8 query heads, how many key-value heads does multi-query attention use, and how many standard multi-head attention? What is grouped-query attention between them?
3. Why is the output layer's matrix multiplication counted in the training FLOPs even though its weights are tied to the embedding, whose lookup costs nothing?
4. Derive the $6N$ rule for training FLOPs per token from Chapter 1.2's result that the backward pass of a matrix multiplication costs twice the forward pass.
5. The model is initialised with standard deviation 0.02 and residual projections scaled by $1/\sqrt{2L}$. Estimate the standard deviation of the residual stream after all eight blocks at initialisation.

COMPLETION CHECKLIST
- [ ] I can draw the transformer and explain the purpose of each component.
- [ ] I can explain rotary embeddings, grouped-query attention and SwiGLU.
- [ ] I can count a transformer's parameters and training FLOPs from its configuration.
- [ ] I can test a transformer for causality, relative positions and capacity.
- [ ] Lab 2.4.1–2.4.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.5

Pretraining

TIME	8 hours, and about five hours of the GPU's time
YOU BUILD	A pipeline that trains the tokeniser on 2.2 GB of stories and encodes them into a file of 536 million tokens; a training loop with warm-up, cosine decay, gradient clipping, accumulation and mixed precision; training throughput measured four ways; and Chapter 2.4's model pretrained on 300 million tokens, from noise to coherent stories.
NEEDS	Chapters 1.4, 2.1 and 2.4.

WHY IT MATTERS

In pretraining a language model acquires nearly everything it knows, by predicting the next token over a large body of text for as long as compute allows. Fine-tuning, preference training and tools, in Unit 3, adjust what pretraining built. The algorithm is Unit 1's training loop, but at a scale where handling the data, keeping the optimisation stable, using the hardware fully and surviving interruptions matter as much as the model. This chapter trains the course's model in full, in an afternoon, and produces the model that the rest of this unit examines and Unit 4 makes fast.

SKIP TEST

1. How many tokens does each step see with a batch of 32 sequences of 512 tokens, and how many steps make 300 million tokens?
2. Why warm up the learning rate at the start of training with Adam?
3. What does gradient clipping protect against, and what does it cost when nothing is wrong?
4. When is accumulating gradients over several small batches exactly equivalent to one large batch?

If all four are easy, read the sections on throughput and on the run, and do Labs 2.5.1 and 2.5.3.

Core ideas

The data

TinyStories, from Ronen Eldan and Yuanzhi Li, is a collection of short stories written by GPT-4 in the vocabulary of a three- or four-year-old, each prompted with a few required words and features such as dialogue or a twist. Its training split holds 2.72 million stories in 2.2 GB, and its validation split 27,630 more. The language is simple and the world it describes is small, so a model of 28 million parameters can learn both well enough to write coherent stories, and the effects of every choice in this unit are visible on a single machine.

`u2.prepare` trains Chapter 2.1's tokeniser on the training split with a vocabulary of 8,192, then encodes both splits into flat files of 16-bit token ids, with an end-of-text token between stories. Both steps run on every CPU core: the file is cut into pieces at story boundaries, each process counts or encodes its pieces, and the results are combined in order.

Encoding a file in parallel, in pieces that end at story boundaries — code/aif/corpus.py

```
def _encode(args) -> np.ndarray:
    path, start, end, tokenizer = args
    tok = bpe.BPE.load(tokenizer)
    eot = tok.special[EOT]
    out = []
    for doc in documents(path, start, end):
        out.extend(tok.encode(doc))
        out.append(eot)
    return np.array(out, dtype=np.uint16)


def encode_file(path: Path, tokenizer: Path, out: Path, workers: int | None = None) -> int:
    workers = workers or os.cpu_count()
    jobs = [(path, a, b, tokenizer) for a, b in ranges(path, workers * 4)]
    with ProcessPoolExecutor(workers) as pool, open(out, "wb") as f:
        n = 0
        for ids in pool.map(_encode, jobs):
            ids.tofile(f)
            n += len(ids)
    return n
```

Train the tokeniser, then encode both splits — code/u2/prepare.py

```
if not tokenizer.exists():
    start = time.perf_counter()
    counts = corpus.count_file(train_txt)
    tok = bpe.train(counts, args.vocab, special=[corpus.EOT])
    tok.save(tokenizer)
    print(
        f"tokeniser: {len(counts):,} distinct chunks, {args.vocab} tokens "
        f"in {time.perf_counter() - start:.0f} s"
    )
tok = bpe.BPE.load(tokenizer)

stats = {}
for split, path in (("train", train_txt), ("valid", valid_txt)):
    start = time.perf_counter()
    n = corpus.encode_file(path, tokenizer, out / f"{split}-{args.vocab}.bin")
    size = path.stat().st_size
    stats[split] = {"tokens": n, "bytes": size}
    print(
        f"{split}: {size / 1e6:,.0f} MB of text → {n / 1e6:,.1f}M tokens "
        f"({size / n:.2f} bytes per token) in {time.perf_counter() - start:.0f} s"
    )
```

The corpus, prepared

```
$ uv run -m u2.prepare
tokeniser: 59,732 distinct chunks, 8192 tokens in 23 s
train: 2,228 MB of text → 536.2M tokens (4.15 bytes per token) in 25 s
valid: 23 MB of text → 5.4M tokens (4.16 bytes per token) in 2 s
the first complete validation story begins: 'Once upon a time, in a warm and sunny place,
    there was a big pit. A little boy named Tom liked to play near the pit.'
```

Batches

A training batch is 32 windows of 513 consecutive tokens taken at random positions in the token file; the first 512 are the input and the last 512, shifted by one, the targets. The file is opened as a NumPy *memory map*, so the operating system reads only the windows that are used and the 1 GB file never has to fit in memory at once.

Random training windows, and fixed validation windows — code/aif/train.py

```
class TokenFile:
    def __init__(self, path: Path, seq: int):
        self.ids = np.memmap(path, dtype=np.uint16, mode="r")
        self.seq = seq

    def batch(
        self, size: int, rng: np.random.Generator, device
    ) -> tuple[torch.Tensor, torch.Tensor]:
        starts = rng.integers(0, len(self.ids) - self.seq - 1, size)
        rows = np.stack([self.ids[s : s + self.seq + 1] for s in starts]).astype(np.int64)
        rows = torch.from_numpy(rows).to(device, non_blocking=True)
        return rows[:, :-1], rows[:, 1:]

    def fixed(self, count: int, device) -> list[tuple[torch.Tensor, torch.Tensor]]:
        step = (len(self.ids) - self.seq - 1) // count
        rows = np.stack(
            [self.ids[i * step : i * step + self.seq + 1] for i in range(count)]
        ).astype(np.int64)
        rows = torch.from_numpy(rows).to(device)
        return [(rows[i : i + 16, :-1], rows[i : i + 16, 1:]) for i in range(0, count, 16)]
```

Windows cross story boundaries freely. The model sees end-of-text tokens and learns that a new story begins after one, and its attention can look across the boundary into the previous, unrelated story, which it learns to ignore. Production pipelines often pack documents into windows and mask attention at their boundaries instead, which matters more for long documents of mixed kinds.

The run draws 300 million tokens' worth of windows from a file of 536 million, so it sees each token about half a time on average and almost every window is new to it. Its training loss is then a fair estimate of its loss on unseen text, and it cannot overfit in the sense of Chapter 1.6, since there is more data than the compute budget can use. Validation uses 64 fixed windows, evenly spaced through the validation file, the same at every evaluation, so that successive measurements differ only because the model changed.

The optimiser and its schedule

The optimiser is Chapter 1.4's AdamW with $\beta_1 = 0.9$ and $\beta_2 = 0.95$, and weight decay of 0.1 applied to the weight matrices but not to the normalisation gains, which set scales that decay would pull towards zero. The smaller β_2 is standard for transformers. The second-moment estimate averages over roughly the last $1/(1-\beta_2)$ steps, so it follows changes in the gradients' scale within about 20 steps instead of about 1,000. That avoids the oversized updates that a stale, too-small estimate produces.

The model, the compiled step and the optimiser — code/u2/pretrain.py

```
shape = dict(PRESETS[args.preset])
if args.d_model:
    heads = args.d_model // args.head_dim
    shape.update(d_model=args.d_model, n_heads=heads, n_kv_heads=kv_heads(heads))
if args.layers:
    shape["n_layers"] = args.layers
if args.kv_heads:
    shape["n_kv_heads"] = args.kv_heads
config = gpt.Config(
    vocab_size=tok.vocab_size, max_seq=args.seq, d_ff=ffn_width(shape["d_model"]), **shape
)
model = gpt.GPT(config).to(dev)
step_model = model if args.no_compile else torch.compile(model)
optimizer = torch.optim.AdamW(
    train.param_groups(model, args.wd), lr=args.lr, betas=(0.9, args.beta2)
)
dtype = {"fp32": None, "fp16": torch.float16, "bf16": torch.bfloat16}[args.dtype]
scaler = torch.amp.GradScaler(dev.type, enabled=args.dtype == "fp16")
tokens_per_step = args.batch * args.accum * args.seq
total_steps = int(args.tokens // tokens_per_step)
```

The learning rate rises linearly over the first 500 steps, the *warm-up*, then follows a cosine down to a tenth of its peak at the end of the run. Warm-up is needed because Adam normalises its updates. At the first

step every weight moves by the full learning rate, whatever its gradient, while the network is still at its initialisation and the moment estimates rest on a single batch. Starting small lets the estimates settle and the network's scales adjust before the steps grow.

Linear warm-up, then cosine decay to a floor — code/aif/train.py

```
def lr_at(step: int, total: int, peak: float, warmup: int, floor: float = 0.1) -> float:
    if step < warmup:
        return peak * (step + 1) / warmup
    progress = min(1.0, (step - warmup) / max(1, total - warmup))
    return peak * (floor + (1 - floor) * 0.5 * (1 + math.cos(math.pi * progress)))
```

The peak learning rate is the most important setting in the run, and the cheapest way to choose it is a set of short runs. Three runs of 6 million tokens each, with peaks of 10^{-3}, 2×10^{-3} and 3×10^{-3} and schedules scaled to their length, reached validation losses of 2.251, 2.198 and 2.246 nats. The full run uses 2×10^{-3}.

> **PITFALL**
> Compare settings at the end of runs of the same length. A short run's schedule decays sooner, so midway through a long run its loss is higher than a short run's at the same point, and every long schedule judged mid-run looks worse than it will finish. The best learning rate also drifts down slowly as runs get longer, so a short sweep only approximates the best rate for a long run.

The loop

One optimiser step: accumulate, unscale, clip, step — code/u2/pretrain.py

```
eot = tok.special["◁endoftext▷"]
started, seen = time.perf_counter(), 0
while step < total_steps:
    lr = train.lr_at(step, total_steps, args.lr, args.warmup)
    for group in optimizer.param_groups:
        group["lr"] = lr
    total = 0.0
    for _ in range(args.accum):
        x, y = train_ids.batch(args.batch, rng, dev)
        with torch.autocast(dev.type, dtype=dtype, enabled=dtype is not None):
            _, loss = step_model(x, y)
        scaler.scale(loss / args.accum).backward()
        total += loss.item() / args.accum
    scaler.unscale_(optimizer)
    norm = torch.nn.utils.clip_grad_norm_(model.parameters(), args.clip or float("inf"))
    scaler.step(optimizer)
    scaler.update()
    optimizer.zero_grad(set_to_none=True)
    step += 1
    seen += tokens_per_step
```

Three additions turn Unit 1's loop into one fit for long runs.

Gradient clipping rescales the whole gradient whenever its norm exceeds 1.0. An unusual batch can produce a gradient many times larger than normal, and one such step can undo hours of training. Clipping limits the size of that step. When nothing is wrong the norm stays below the threshold and clipping does nothing.

Gradient accumulation runs several smaller batches, adds their gradients, and steps once. Since each loss is divided by the number of batches, and every batch holds the same number of tokens, the summed gradient is exactly the gradient of the mean loss over the combined batch. A model with no batch-dependent layers trains identically and needs only the memory of the smaller batch.

Mixed precision runs the forward pass in a 16-bit format inside `torch.autocast` while keeping the weights in float32. With float16, whose range is narrow, the loss is first multiplied by a large factor so that small gradients do not round to zero, and the gradients are divided by it again before clipping. `GradScaler` does both and adjusts the factor when it overflows. Chapter 4.3 examines the formats.

The program also saves the model, the optimiser's state, the random generator's state and the step number every 2,000 steps, and `--resume` continues from them exactly. A five-hour run can be interrupted by a crash, a power cut or a laptop going to sleep, and a run that cannot resume is then lost.

Throughput

The speed of training is measured in tokens per second, and its efficiency in *model FLOPs utilisation*: the FLOPs the model needs per token, 179 million here, times the tokens per second, as a fraction of the GPU's peak. The program trains the model for a few steps in four ways and times each:

Timing training steps, after five steps of warm-up and compilation — code/u2/throughput.py

```
def tokens_per_second(
    dev, dtype, compiled: bool, batch: int, steps: int
) -> tuple[float, float]:
    device.seed(0)
    model = gpt.GPT(gpt.Config()).to(dev)
    step_model = torch.compile(model) if compiled else model
    opt = torch.optim.AdamW(train.param_groups(model, 0.1), lr=1e-3, betas=(0.9, 0.95))
    scaler = torch.amp.GradScaler(dev.type, enabled=dtype == torch.float16)
    tokens = train.TokenFile(data.DATA / "tinystories" / "train-8192.bin", 512)
    rng = np.random.default_rng(0)
    times = []
    for _ in range(steps + 5):
        x, y = tokens.batch(batch, rng, dev)
        start = time.perf_counter()
        with torch.autocast(dev.type, dtype=dtype, enabled=dtype is not None):
            _, loss = step_model(x, y)
        scaler.scale(loss).backward()
        scaler.unscale_(opt)
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(opt)
        scaler.update()
        opt.zero_grad(set_to_none=True)
        device.sync(dev)
        times.append(time.perf_counter() - start)
    return batch * 512 / float(np.median(times[5:])), loss.item()
```

Training throughput on the M2 Pro

```
$ uv run -m u2.throughput
Chapter 2.5's model, batch 32 x 512 tokens, on mps:
precision mode        tokens/s  TFLOP/s  of peak
float32   eager          9,364     1.68      25%
float32   compiled      18,107     3.25      48%
float16   compiled      19,046     3.42      50%
bfloat16  compiled      12,428     2.23      33%
```

Compilation nearly doubles throughput, from 9,400 to 18,100 tokens per second. The matrix multiplications are the same in both, and the gain comes from fusing the element-wise operations between them, which Chapter 4.2 examines. Mixed precision gains little on this machine: float16 is 5% faster than float32, and bfloat16, for which the M2 Pro's GPU has no arithmetic, is a third slower. Chapter 4.3 explains why, and why the same flag gives several times the speed on an NVIDIA GPU with tensor cores. The run therefore uses compiled float32. At 3.25 TFLOP/s it uses 48% of the GPU's nominal float32 peak, within the range of about 35% to 55% that large training runs on thousands of GPUs typically report.

The run

```
uv run -m u2.pretrain
```

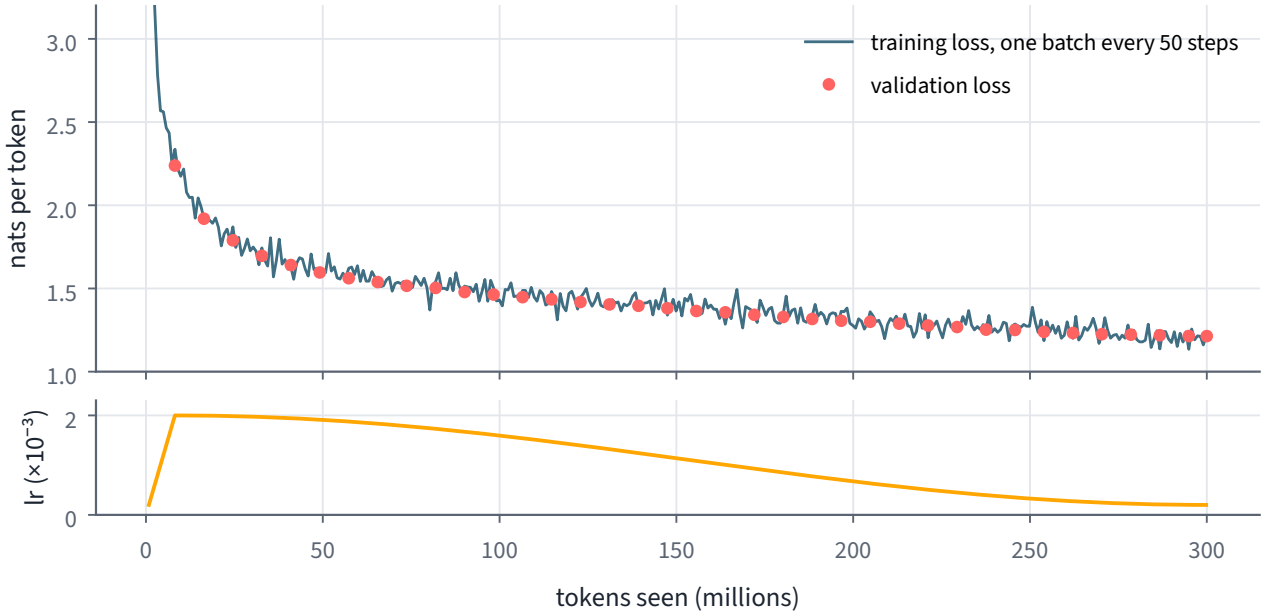


Figure 2.5.1. Training loss every 50 steps and validation loss every 500, against tokens seen, with the learning-rate schedule below. The two losses track each other throughout, because with less than one pass over the data the model cannot overfit.

The start of the run

```
$ uv run -m u2.pretrain
27.8M parameters, 179 MFLOP per token, 18310 steps of 16384 tokens
step    500  train 2.337  val 2.239 (0.777 bits/byte)  17,886 tokens/s
step   1000  train 1.923  val 1.919 (0.666 bits/byte)  17,449 tokens/s
step   1500  train 1.870  val 1.789 (0.621 bits/byte)  17,487 tokens/s
step   2000  train 1.744  val 1.696 (0.589 bits/byte)  17,575 tokens/s
```

The end of the run

```
step  17000  train 1.199  val 1.222 (0.424 bits/byte)  17,555 tokens/s
step  17500  train 1.137  val 1.219 (0.423 bits/byte)  17,569 tokens/s
step  18000  train 1.135  val 1.214 (0.421 bits/byte)  17,581 tokens/s
step  18310  train 1.260  val 1.214 (0.421 bits/byte)  17,588 tokens/s
```

The run took 4 hours and 44 minutes and ended at a validation loss of 1.214 nats per token, 0.421 bits per byte. That is about half of both the 5-gram model's 2.474 nats and the neural model's 2.366 from Chapter 2.2, and well below the 0.70 bits per byte that SmolLM2-135M, five times larger and trained on the web, scores on the same text in Chapter 3.1. The curve has the usual shape of a pretraining run. The loss falls steeply over the first few million tokens, as the model learns the vocabulary and the commonest phrases, then declines slowly for the rest of the run as it learns grammar, characters and plots. The last stretch, as the learning rate falls to its floor, still gains a little.

From noise to stories

The program logs a story sampled from the model at every evaluation. Reading them in order shows what is learned when:

The logged samples, in order — code/u2/stories.py

```python
log = [r for r in train.read_log(data.RUNS / args.model / "log.jsonl") if "sample" in r]
for r in [r for r in log if r["step"] in args.steps] + log[-1:]:
    text = " ".join(r["sample"].split())[:260]
    print(f"after {r['tokens'] / 1e6:5.0f}M tokens (loss {r['val_loss']:.2f}): {text}...")
```

Samples during training, then two stories from the finished model

```
$ uv run -m u2.stories
after     8M tokens (loss 2.24): were pushed and Sam. They liked to make things for hands
```

```
    and toys. One day, they found a big box. They wondered what was inside. They did not
    know what was inside the box. They pushed a button and it broke. They pulled it in. The
    box broke, and it broke into p...
after    33M tokens (loss 1.70): One day, a boy named Tim found a big hiding place. He was
    very excited. He was hungry. He wanted to find out why it was empty. Tim went to his
    friend, Sam. "Sam, I found a big, yummy worm. I found it outside," he said. "Wow,
    that's a big, yummy worm. But I can...
after   131M tokens (loss 1.40): One day, a boy named Tim found a big hiding spot under his
    bed. He was very happy and wanted to play. But, when he looked around, he saw that it
    was dark outside. Tim felt uncomfortable because it was gloomy outside. He called out,
    "Boom, where is the sun?" To...
after   300M tokens (loss 1.21): One day, a boy named Tim found a big hiding place. He was
    very excited. He wanted to show his mom. But, when he got there, it was very noisy.
    There was a big dog outside. The big dog was his pet. Tim was scared. The dog barked
    and Tim felt sad. He wanted him t...
Tom and his sister found a map in the attic. It had a big X on it. They decided to follow
the map and see what was down the hill. They walked and walked until they saw the X. They
walked and walked until they reached a big hole. Tom looked inside and saw something shiny.
He reached his hand inside and saw a red liquid. It was a sparkly key. He showed it to his
sister. "Look, I found a treasure!" he said. "Wow, Tom, that's amazing!" she said. "What is
that?" Tom looked at the key and smiled. He took out a small piece of paper. He showed it to
his sister. "Look, I found a treasure!" he said. He put the key in the lock and turned it.
The key clicked shut. Tom and his sister pulled it. The lock opened. There was a loud click.
"Yay, we unlocked the door, Tom! We are so lucky!" he said. Tom and his sister were very
excited. They put the key back in the hole and ran to the porch. They wanted to show their
mom and dad their treasure. They were very happy.
The little robot was afraid of the dark. It did not want to play much. It wanted to go home.
The robot said, "No, I don't want to go. I want to stay here." The little robot was sad. It
did not understand why the dark was so scary. But it could not leave the little robot. The
robot went home, still scared. The end.
```

Every evaluation samples with the same random seed, so the samples begin alike and the differences between them come from the model. After 8 million tokens it writes grammatical sentences with little memory: it starts mid-sentence, repeats itself and loses track of what it has said. By 33 million it has names, correctly punctuated dialogue and the opening formulas of the genre, but contradicts itself from one sentence to the next. By 131 million a scene holds together for a paragraph, with feelings that follow from events. The final model keeps a story's characters and objects for its whole length and brings it to an ending. The treasure-map story follows its prompt from the X on the map to a hole, a key and a door. Its failures are typical of a small model: details that contradict each other ("a red liquid. It was a sparkly key"), a repeated line of dialogue, and a lock that clicks shut and then opens.

> **KEY IDEA**
> Pretraining is next-token prediction at scale: random windows from a large corpus, AdamW with warm-up, decay and clipping, and as many tokens per second as the hardware allows. With less than one pass over the data, the training loss is a fair estimate of the loss on new text, and the compute budget, which runs out before the data does, limits the model.

> **IN PYTORCH**
> `torch.compile`, `torch.autocast`, `torch.amp.GradScaler` and `torch.nn.utils.clip_grad_norm_` are this chapter's tools, and `torch.optim.lr_scheduler` offers schedules ready-made. Hugging Face's `datasets` library streams corpora too large for one disk. Andrej Karpathy's nanoGPT and llm.c are compact, readable implementations of the same training at larger scale, and torchtitan and Megatron-LM are what large runs use.

Reading

- Ronen Eldan and Yuanzhi Li, "TinyStories: How Small Can Language Models Be and Still Speak Coherent English?", 2023.

- Aakanksha Chowdhery and others, “PaLM: Scaling Language Modeling with Pathways”, 2022: section 4 defines model FLOPs utilisation.
- Mitchell Wortsman and others, “Small-Scale Proxies for Large-Scale Transformer Training Instabilities”, ICLR 2024: what warm-up, learning rate and normalisation do to stability.
- Andrej Karpathy, nanoGPT, 2023, and “Let’s reproduce GPT-2 (124M)”, 2024: a complete pretraining run, explained.

Labs

LAB 2.5.1 Your run

1.5 hours of work, and the run

Goal. Pretrain the model you will use for the rest of the course.

Code. `code/u2/pretrain.py`

1. Run `uv run -m u2.pretrain`. On a machine much slower than the M2 Pro, use `--preset small --tokens 100e6` instead and expect weaker stories.
2. Plot your training and validation losses from `runs/tinystories-28m/log.jsonl`, and read the samples at a few points.

DONE WHEN

- [ ] Your run finished, or resumed and finished, and saved `model.pt`.
- [ ] With the base preset, your final validation loss is within 0.03 nats of this chapter’s.

EVIDENCE FOR THE NOTEBOOK

- The plot and the final line of the log.

LAB 2.5.2 A learning-rate sweep

2 hours

Goal. Choose a learning rate the way large runs do.

Code. `code/u2/pretrain.py`

1. Train the base model on 20 million tokens with peak learning rates of 5×10^{-4}, 10^{-3}, 2×10^{-3}, 3×10^{-3} and 5×10^{-3}, each with its own name.
2. Plot the final validation loss against the learning rate on a logarithmic axis.

DONE WHEN

- [ ] Your plot has five points and a clear minimum, or you extend the range until it does.
- [ ] You can state the range of learning rates within 0.02 nats of the best.

EVIDENCE FOR THE NOTEBOOK

- The plot.

LAB 2.5.3 What keeps it stable

2 hours

Goal. Find which of warm-up, clipping and β_2 this run depends on.

Code. `code/u2/pretrain.py`

1. At a peak learning rate of 5×10^{-3} and 20 million tokens, train once as is, once without warm-up (`--warmup 0`), once without clipping (`--clip 0`) and once with $\beta_2 = 0.999$ (`--beta2 0.999`), each with its own `--name`.
2. Plot the loss and the gradient norm from each log.

DONE WHEN

- [ ] Your table has the four final losses.

- [ ] You can point to a loss spike, or its absence, and explain it from the gradient norms.

EVIDENCE FOR THE NOTEBOOK
- The table and the plots.

LAB 2.5.4 Checking gradient accumulation — 1 hour

Goal. Show that accumulation reduces memory and leaves the gradients unchanged.

Code. `code/u2/pretrain.py`

1. Make the loop print the loss at every step. Run 20 steps, `--tokens 327680`, with `--batch 32` and with `--batch 8 --accum 4`, both with `--no-compile` and the same seed; the random generator then draws the same windows for both.
2. Compare their losses step by step, and their speed and memory.

DONE WHEN
- [ ] The two runs' losses agree to within rounding, or you can explain any difference.
- [ ] You have both runs' tokens per second and peak memory.

EVIDENCE FOR THE NOTEBOOK
- The comparison.

Problem set

1. The run draws 586,000 windows of 513 tokens from 536 million possible starting positions. About how many windows are drawn twice?
2. Training needs 179 MFLOP per token. How long would the run's 300 million tokens take on a GPU sustaining 400 TFLOP/s?
3. Show that summing the gradients of k batches' losses, each divided by k, gives the gradient of the mean loss over the combined batch when the batches are the same size. Name two things that would break the equality.
4. At Adam's first step, with bias correction and ε negligible, how far does each weight move? What does that suggest about the first steps of training at a high learning rate?
5. The validation loss is measured on 64 windows of 512 tokens, and per-token losses have a standard deviation of about 1.5 nats. Estimate the standard error of the validation loss if tokens were independent, and say whether the true error is larger or smaller.

COMPLETION CHECKLIST
- [ ] I can prepare a corpus for training: a tokeniser, a token file and a held-out split.
- [ ] I can write a training loop with warm-up, decay, clipping, accumulation and mixed precision, and explain each part.
- [ ] I can measure training throughput and model FLOPs utilisation.
- [ ] I can run, monitor and resume a long training run, and read its curves.
- [ ] Lab 2.5.1–2.5.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.6

Scaling laws

TIME	6 hours, and about three hours of the GPU's time
YOU BUILD	Fifteen training runs at three compute budgets, five model sizes at each; the best size and loss at each budget, found by fitting isoFLOP curves; power laws fitted to the optima; and a prediction of the best loss at a budget four times larger, checked by training the model it recommends.
NEEDS	Chapters 2.4 and 2.5.

WHY IT MATTERS

A large model is trained once, at great cost, and its size and the amount of data it sees must be chosen before the run starts. Scaling laws make that choice from cheap experiments. Across many orders of magnitude, a language model's loss falls as a smooth power law in the compute spent on it, so a sweep of small runs predicts a large one. These laws were used to choose the sizes of the models in use today and the number of tokens each was trained on. This chapter fits a scaling law on the course's data and model, and tests it.

SKIP TEST

1. About how many FLOPs does training a model of N parameters on D tokens take, and where does the factor come from?
2. What is an isoFLOP curve, and why does it have a minimum?
3. What did the Chinchilla study conclude about dividing compute between parameters and tokens?
4. Why are many widely used models trained on far more tokens than is compute-optimal?

If all four are easy, read the sections on the sweep and on what the laws are for, and do Labs 2.6.1 and 2.6.3.

Core ideas

Compute

Training a model with N parameters on D tokens costs about

$$C \approx 6ND$$

FLOPs. The forward pass multiplies and adds each parameter once per token, 2 FLOPs. The backward pass computes gradients with respect to both the activations and the parameters, twice the forward work. Attention adds a term that grows with the sequence length, and Chapter 2.4's `train_flops_per_token` counts it, with the embedding counted once as the output layer's matrix multiplication. For a fixed budget C, a larger model sees proportionally fewer tokens.

Power laws

Jared Kaplan and colleagues found in 2020 that a transformer's loss, when nothing else limits it, falls as a power of the parameters, of the data and of the compute, each over several orders of magnitude: a straight line on logarithmic axes. Jordan Hoffmann and colleagues, training more than 400 models for the Chinchilla study of 2022, fitted the loss of a model of N parameters trained on D tokens as

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}},$$

with E the irreducible loss, the entropy of the text itself, and two terms that shrink as the model and the data grow. Minimising it under the constraint $C = 6ND$ gives an optimal size and token count that both grow as a power of the budget. Chinchilla's exponents were both close to $1/2$, so when the compute doubles, the model and its data should each grow by about $\sqrt{2}$, at about 20 tokens per parameter.

IsoFLOP curves

The most direct way to find the optimal size is to fix a budget, train several sizes on it, each with $D = C/(6N)$ tokens, and see which reaches the lowest loss. A model that is too small cannot represent what the data contains, and one that is too large does not see enough of it. Plotted against $\log N$, the losses at a fixed budget form a valley, an *isoFLOP curve*, and a parabola fitted to it locates the bottom. Repeating at several budgets gives the optimal size $N^*(C)$ and loss $L^*(C)$, and straight lines fitted to their logarithms give the power laws.

Nine shapes, from 0.6M to 19M parameters, and the five trained at each budget — code/u2/scaling.py

```
SHAPES = [
    (64, 2),
    (96, 2),
    (128, 3),
    (160, 4),
    (192, 4),
    (256, 4),
    (320, 5),
    (384, 6),
    (448, 7),
]
BUDGETS = {1e14: SHAPES[0:5], 4e14: SHAPES[1:6], 1.6e15: SHAPES[2:7]}
```

The runs use sequences of 256 tokens and attention heads of 32 dimensions so that the smallest models remain sensible transformers, and every run is Chapter 2.5's training with its own warm-up and cosine schedule, ending at its own budget. Kaplan's study used a schedule of fixed length for runs of every length, which made short runs look worse than they are and pushed its recommended models to be larger and its data smaller. Chinchilla corrected that bias.

One run of the sweep, at a given shape and budget — code/u2/scaling.py

```
def run(d: int, layers: int, budget: float) -> float:
    name = run_name(d, layers, budget)
    log = data.RUNS / name / "log.jsonl"
    if not (data.RUNS / name / "model.pt").exists():
        tokens = budget / gpt.train_flops_per_token(config(d, layers), SEQ)
        done = subprocess.run(
            [
                sys.executable,
                "-m",
                "u2.pretrain",
                "--name",
                name,
                "--d-model",
                str(d),
                "--layers",
                str(layers),
                "--head-dim",
                str(HEAD_DIM),
                "--seq",
                str(SEQ),
                "--batch",
                str(BATCH),
                "--tokens",
                str(tokens),
                "--lr",
                "3e-3",
                "--warmup",
                "200",
                "--eval-every",
                "100000",
```

```
                "--eval-windows",
                "512",
                "--ckpt-every",
                "100000",
            ],
            capture_output=True,
            text=True,
        )
        check.expect(f"{name} trained", done.returncode == 0, done.stderr[-500:])
    return [r for r in train.read_log(log) if "val_loss" in r][-1]["val_loss"]
```

Parabolas through each budget's losses, then power laws through their minima — code/u2/scaling.py

```
optima = []
for budget in BUDGETS:
    rows = np.array([(math.log(n), loss) for b, n, _, loss in points if b == budget])
    a, b, c0 = np.polyfit(rows[:, 0], rows[:, 1], 2)
    best_log_n = -b / (2 * a)
    optima.append((budget, math.exp(best_log_n), c0 - b * b / (4 * a)))
logs = np.log([[c, n, loss] for c, n, loss in optima])
n_slope, n_icept = np.polyfit(logs[:, 0], logs[:, 1], 1)
l_slope, l_icept = np.polyfit(logs[:, 0], logs[:, 2], 1)
predicted_n = math.exp(n_icept + n_slope * math.log(args.check))
predicted_loss = math.exp(l_icept + l_slope * math.log(args.check))
```

The sweep

```
uv run -m u2.scaling
```

Fifteen runs, the fitted laws, and the prediction checked

```
$ uv run -m u2.scaling
 budget (FLOPs)  d_model  layers  parameters    tokens  val loss
        1.0e+14       64       2       0.62M     25.4M     2.483
        1.0e+14       96       2       0.98M     16.1M     2.422
        1.0e+14      128       3       1.64M      9.6M     2.314
        1.0e+14      160       4       2.42M      6.5M     2.365
        1.0e+14      192       4       3.20M      4.9M     2.418
        4.0e+14       96       2       0.98M     64.6M     2.232
        4.0e+14      128       3       1.64M     38.4M     2.012
        4.0e+14      160       4       2.42M     25.8M     1.970
        4.0e+14      192       4       3.20M     19.6M     1.962
        4.0e+14      256       4       5.05M     12.6M     2.007
        1.6e+15      128       3       1.64M    153.5M     1.881
        1.6e+15      160       4       2.42M    103.3M     1.797
        1.6e+15      192       4       3.20M     78.6M     1.750
        1.6e+15      256       4       5.05M     50.2M     1.729
        1.6e+15      320       5       8.46M     30.1M     1.765
budget 1.0e+14: the best size is about 1.67M parameters, loss 2.343
budget 4.0e+14: the best size is about 2.99M parameters, loss 1.951
budget 1.6e+15: the best size is about 4.95M parameters, loss 1.729
fits: best size grows as C^0.39, best loss falls as C^-0.110
prediction for 6.4e+15 FLOPs: about 8.6M parameters, loss 1.470
trained d_model 320, 5 layers at 6.4e+15 FLOPs: loss 1.561, 6.2% above the prediction
```

Every budget has a clear valley, and its bottom moves to larger models as the budget grows: about 1.7M parameters at 10^{14} FLOPs, 3.0M at 4×10^{14} and 5.0M at 1.6×10^{15} (Figure 2.6.1). The best size grows as $C^{0.39}$, more slowly than Chinchilla's $C^{1/2}$, so the best number of tokens grows faster, from about 6 tokens per parameter at the smallest budget to 7 and then 11, all below Chinchilla's 20. Both differences are expected at this scale. The counts include the embedding table, which with a vocabulary of 8,192 holds 84% of the smallest model's parameters and 31% of the largest's. Every run also uses the same learning rate, which suits some sizes better than others. Labs 2.6.2 and 2.6.4 measure both effects.

The best loss falls as $C^{-0.110}$, and extrapolated to 6.4×10^{15} FLOPs it predicts 1.470 for a model of about 8.6M parameters. The nearest shape, 8.46M parameters, trained at that budget reaches 1.561, 6.2% above the prediction. Each fourfold increase in compute cut the best loss by 17%, then by 11%, and the test run's by

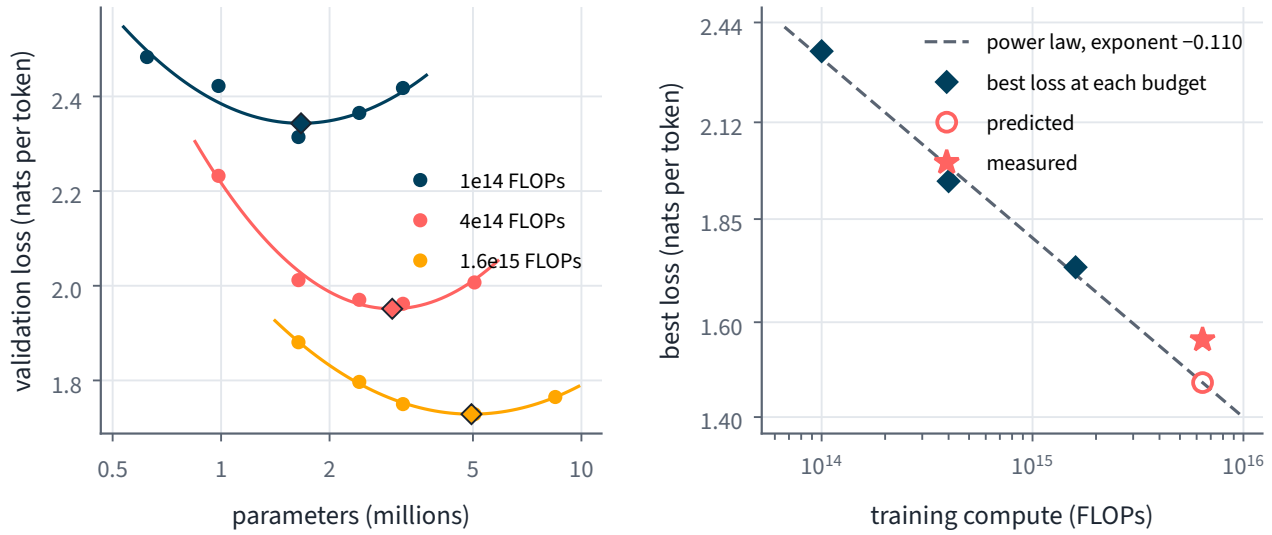


Figure 2.6.1. Left, the isoFLOP curves: validation loss against model size at three budgets, with fitted parabolas and their minima. Right, the best loss at each budget on logarithmic axes, the fitted power law extended to four times the largest budget, and the run that tested it.

10%. The gains shrink as the loss approaches the entropy of the text itself, the floor E of Chinchilla's law, and a straight line on logarithmic axes has no floor. Fitted to all fifteen runs, the three-term law of Lab 2.6.1 puts the floor near 1.27 nats and predicts the test run within about 1%.

What the laws are for

Scaling laws answer the planning question, what size of model for this budget, and they predict the result before it is paid for. Groups that train large models fit laws like this chapter's on small runs of their own data and architecture. They also use them to compare ideas: a change to the architecture or the data that shifts the whole line down is worth the same as a multiple of the compute.

A compute-optimal size minimises the cost of training alone. A model is trained once and then used, sometimes for trillions of tokens, at a cost per token proportional to its size, so a smaller model trained on more data than is compute-optimal can be cheaper over its life. Meta's Llama 3 8B was trained on 15 trillion tokens, nearly 2,000 per parameter. SmolLM2-135M, which Unit 3 uses, was trained on 2 trillion, about 15,000 per parameter. Loss keeps falling with more tokens at a fixed size, only more slowly, and the laws say how slowly.

The laws have limits. They hold for one corpus, tokeniser, architecture and recipe, and must be refitted when any of them changes. They assume fresh data. Repeating data costs little for a few epochs and then quickly stops helping, so the largest runs are now constrained by the supply of good text. They also predict loss and say nothing directly about abilities. Some capabilities appear to arrive suddenly with scale, although much of that suddenness comes from all-or-nothing measures of success that hide steady improvement.

PITFALL

A power law fitted over one or two orders of magnitude can be extrapolated only a short distance. Small errors in the exponent grow with the distance, and effects that the fit never saw, such as running out of unique data or an unstable learning rate at large width, appear at scale. Test a prediction at a budget just beyond the fit, as this chapter does, before trusting it further.

KEY IDEA

Loss falls with compute roughly as a power law, flattening as it nears the entropy of the text. At a fixed budget there is a best model size, which grows as a power of the budget, and a sweep of small runs fits both laws and predicts a larger run. Deployed models are often trained well beyond compute-optimal, because a smaller model is cheaper to use.

IN PRACTICE

`torch.utils.flop_counter.FlopCounterMode` counts the FLOPs of any PyTorch model, a check on hand-derived formulas such as `train_flops_per_token`. `scipy.optimize.curve_fit` fits the three-term law to a set of runs, as Lab 2.6.1 does. Parametrisations such as µP make the best learning rate transfer from small models to large ones, so that a sweep's small runs predict the large one more reliably.

Reading

- Jared Kaplan and others, "Scaling Laws for Neural Language Models", 2020.
- Jordan Hoffmann and others, "Training Compute-Optimal Large Language Models", NeurIPS 2022: the Chinchilla study.
- Niklas Muennighoff and others, "Scaling Data-Constrained Language Models", NeurIPS 2023: what repeated data is worth.
- Greg Yang and others, "Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer", 2022: µP.

Labs

LAB 2.6.1 The three-term law — 2 hours

Goal. Fit Chinchilla's parametric form to your runs.

Code. `code/u2/scaling.py`

1. Fit $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ to all sixteen runs, by least squares on the losses.
2. Minimise the fitted law at a budget of 6.4×10^{15} FLOPs, and compare the optimal size with the isoFLOP method's.

DONE WHEN

- [ ] You have the five fitted constants.
- [ ] The two methods' optimal sizes agree within a factor of two, or you can explain why they do not.

EVIDENCE FOR THE NOTEBOOK

- The fit and the comparison.

LAB 2.6.2 Counting parameters — 1 hour

Goal. See how the choice of what to count changes the law.

Code. `code/u2/scaling.py`

1. Refit the optimal-size power law using the parameters without the embedding.
2. Compare the two exponents and the scatter of the optima about each line.

DONE WHEN

- [ ] You have both exponents.
- [ ] You can say which count gives the cleaner law for these models, and why the embedding matters more for small ones.

EVIDENCE FOR THE NOTEBOOK

- The two fits.

LAB 2.6.3 Smaller and longer — 2 hours

Goal. Measure the price of a model smaller than compute-optimal.

Code. `code/u2/scaling.py`

1. Train a model about a quarter of the predicted optimal size at 6.4×10^{15} FLOPs on the same budget, then on twice and four times the budget.
2. Find the compute at which it matches the optimal model's loss, by interpolation.

DONE WHEN

- [ ] Your table has the three runs.
- [ ] You can state the extra training compute the smaller model needs, and the number of tokens served after which it is the cheaper choice overall.

EVIDENCE FOR THE NOTEBOOK

- The table and the calculation.

LAB 2.6.4 One learning rate for all — 2 hours

Goal. Test an assumption of the sweep.

Code. `code/u2/scaling.py`

1. At a budget of 4×10^{14} FLOPs, train the smallest and the largest of that budget's shapes with learning rates of 10^{-3}, 3×10^{-3} and 10^{-2}.
2. Record the six losses.

DONE WHEN

- [ ] You can say whether the best learning rate depends on size here.
- [ ] You can say which way a size-dependent best rate would bias the fitted exponent.

EVIDENCE FOR THE NOTEBOOK

- The six losses and the argument.

Problem set

1. With $C = 6ND$ and 20 tokens per parameter, what size and token count are compute-optimal at 10^{24} FLOPs?
2. If the best loss falls as $C^{-0.05}$, how much more compute lowers it by 10%?
3. Explain why the backward pass costs about twice the forward pass's FLOPs for a linear layer.
4. Generating a token costs about $2N$ FLOPs. A model will serve 10^{13} tokens. Compare the lifetime compute of a compute-optimal model with that of one half its size trained on enough extra tokens to match its loss, if that takes twice the training compute.
5. Why does a fixed-length learning-rate schedule bias a sweep towards larger models?

COMPLETION CHECKLIST

- [ ] I can count training FLOPs and plan runs at a fixed compute budget.
- [ ] I can find compute-optimal sizes from isoFLOP curves and fit power laws to them.
- [ ] I can predict a larger run's loss from small ones and test the prediction.
- [ ] I can explain why deployed models are trained beyond compute-optimal, and the limits of scaling laws.
- [ ] Lab 2.6.1–2.6.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.7

Decoding

TIME	6 hours
YOU BUILD	A generation loop with greedy choice, beam search, temperature, and top-k, top-p and min-p truncation; eight strategies compared on the same model by probability, repetition and diversity; and a KV cache that makes each new token cost one token's work, timed against generation without it.
NEEDS	Chapters 2.4 and 2.5.

WHY IT MATTERS

A language model gives a probability for every possible next token, and text appears only when something chooses among them, one token at a time. That choice, the *decoding strategy*, changes the output as much as the model does: the same weights write loops, gibberish or good stories depending on it. Choosing token by token is also slow unless the work already done for earlier tokens is kept, which is what the KV cache does. Much of a chat model's style and speed comes from these two pieces.

SKIP TEST

1. Why does greedy decoding of open-ended text tend to repeat itself?
2. What does sampling at temperature T do to the distribution, as T approaches 0 and as it grows above 1?
3. How do top-k, top-p and min-p differ in which tokens they remove?
4. What does a KV cache store, how large is it, and what work does it save?

If all four are easy, read the sections on truncation and on measuring decoding, and do Labs 2.7.2 and 2.7.4.

Core ideas

The generation loop

Generation runs the model on the prompt, chooses a token from the distribution at the last position, appends it, and repeats until an end-of-text token or a length limit. Processing the prompt in one parallel pass is called *prefill*; producing the tokens that follow, one per step, *decode*.

The generation loop, with a KV cache — code/aif/sample.py

```
@torch.no_grad()
def generate(
    model: GPT,
    prompt: list[int],
    max_new: int,
    stop: int | None = None,
    use_cache: bool = True,
    generator=None,
    **sampling,
) -> list[int]:
    device = next(model.parameters()).device
    ids = torch.tensor([prompt], device=device)
    limit = min(model.config.max_seq, len(prompt) + max_new)
    cache = (
        KVCache(model.config, 1, limit, device, next(model.parameters()).dtype)
```

```
        if use_cache
        else None
    )
    logits = model(ids, cache=cache)[:, -1]
    out = list(prompt)
    while len(out) < limit:
        token = next_token(logits, generator=generator, **sampling)
        out.append(int(token))
        if stop is not None and out[-1] == stop:
            break
        if cache is not None:
            logits = model(token.view(1, 1), cache=cache)[:, -1]
        else:
            logits = model(torch.tensor([out], device=device))[:, -1]
    release(device)
    return out[len(prompt) :]
```

Greedy choice and beam search

The simplest rule takes the most probable token every time. *Greedy* decoding is deterministic and often good for short answers with one right reply, but it does not find the most probable sequence, since a likely token now can lead to unlikely ones later. *Beam search* searches further: it keeps the w partial sequences with the highest total log-probability, extends each by every token, keeps the best w of the results, and stops when w sequences have ended.

Beam search over a batch of w sequences, sharing one KV cache — code/aif/sample.py

```
@torch.no_grad()
def beam_search(
    model: GPT,
    prompt: list[int],
    width: int,
    max_new: int,
    stop: int | None = None,
    length_penalty: float = 1.0,
) -> list[int]:
    device = next(model.parameters()).device
    limit = min(model.config.max_seq, len(prompt) + max_new)
    cache = KVCache(model.config, width, limit, device, next(model.parameters()).dtype)
    logits = model(torch.tensor([prompt] * width, device=device), cache=cache)[:, -1]
    beams = [[] for _ in range(width)]
    scores = torch.zeros(width, device=device)
    scores[1:] = float("-inf")
    finished = []
    while len(prompt) + len(beams[0]) < limit and len(finished) < width:
        logp = F.log_softmax(logits.float(), -1) + scores[:, None]
        top = logp.flatten().topk(width)
        rows, tokens = top.indices // logits.shape[-1], top.indices % logits.shape[-1]
        beams = [beams[r] + [int(t)] for r, t in zip(rows.tolist(), tokens.tolist())]
        scores = top.values.clone()
        for i, beam in enumerate(beams):
            if stop is not None and beam[-1] == stop:
                finished.append((float(scores[i]) / len(beam) ** length_penalty, beam))
                scores[i] = float("-inf")
        cache.k, cache.v = cache.k[:, rows], cache.v[:, rows]
        logits = model(tokens.view(-1, 1), cache=cache)[:, -1]
    finished += [
        (float(s) / len(b) ** length_penalty, b)
        for s, b in zip(scores.tolist(), beams)
        if s != float("-inf")
    ]
    release(device)
    return max(finished)[1]
```

For open-ended text, finding more probable sequences is not the goal. Human writing is not the most probable continuation of itself, because it contains unlikely tokens at a steady rate. The highest-probability paths through a model are bland and, given enough length, drift into loops, because once a phrase has

repeated, repeating it again becomes the likeliest continuation. Beam search suits tasks with a single correct output, such as translation, and suits stories poorly, since they should differ every time they are told.

Sampling and temperature

Sampling draws the next token at random from the model's distribution, so every run differs and the text has the variety of the training data. *Temperature* T reshapes the distribution before drawing:

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

At $T = 1$ it is the model's own distribution. Below 1 it sharpens towards the most likely tokens, and as $T \to 0$ sampling becomes greedy. Above 1 it flattens towards uniform.

Choosing one token: greedy at zero temperature, otherwise filter and sample — code/aif/sample.py

```
def next_token(
    logits: torch.Tensor, temperature: float = 1.0, generator=None, **filters
) -> torch.Tensor:
    if temperature == 0.0:
        return logits.argmax(-1)
    logits = filter_logits(logits / temperature, **filters)
    probs = logits.float().softmax(-1)
    return torch.multinomial(probs, 1, generator=generator).squeeze(-1)
```

Pure sampling has a different weakness. A model spreads a little probability over thousands of unlikely tokens. Each is rare, but together they are drawn often enough that a long story will include a few, and one bad token can derail everything after it.

Truncation

Truncation removes the tail before sampling. *Top-k* keeps the k most likely tokens, a fixed number whatever the distribution's shape: too many when the model is sure, too few when many continuations are reasonable. *Top-p*, or nucleus sampling, keeps the smallest set of most likely tokens whose probabilities sum to at least p, which adapts to the shape. *Min-p* keeps every token whose probability is at least a fraction of the most likely token's, so it cuts hard when the model is confident and loosely when it is not, and stays coherent at higher temperatures.

Top-k, top-p and min-p, as masks on the logits — code/aif/sample.py

```
def filter_logits(
    logits: torch.Tensor, top_k: int = 0, top_p: float = 1.0, min_p: float = 0.0
) -> torch.Tensor:
    if top_k > 0:
        kth = torch.topk(logits, top_k, dim=-1).values[..., -1:]
        logits = logits.masked_fill(logits < kth, float("-inf"))
    if top_p < 1.0:
        sorted_logits, order = torch.sort(logits, descending=True, dim=-1)
        probs = sorted_logits.softmax(-1)
        outside = probs.cumsum(-1) - probs > top_p
        logits = logits.scatter(-1, order, sorted_logits.masked_fill(outside, float("-inf")))
    if min_p > 0.0:
        probs = logits.softmax(-1)
        logits = logits.masked_fill(
            probs < min_p * probs.max(-1, keepdim=True).values, float("-inf")
        )
    return logits
```

Measuring decoding

Three measures summarise a strategy, and none of them measures quality directly. The average log-probability per generated token under the model says how likely the text is. The fraction of each sample's four-token sequences that repeat earlier ones within it says how loopy it is. The fraction of distinct three-token sequences across ten samples says how varied the samples are from each other.

Repetition within a sample, and diversity across samples code/u2/decode.py

```python
def repetition(ids: list[int], n: int = 4) -> float:
    grams = [tuple(ids[i : i + n]) for i in range(len(ids) - n + 1)]
    return 1 - len(set(grams)) / max(1, len(grams))


def distinct(samples: list[list[int]], n: int = 3) -> float:
    grams = [tuple(s[i : i + n]) for s in samples for i in range(len(s) - n + 1)]
    return len(set(grams)) / max(1, len(grams))
```

Eight strategies, each applied to the same prompt code/u2/decode.py

```python
strategies = {
    "greedy": dict(temperature=0.0),
    "beam search, width 4": None,
    "temperature 0.7": dict(temperature=0.7),
    "temperature 1.0": dict(temperature=1.0),
    "temperature 1.5": dict(temperature=1.5),
    "top-k 40": dict(temperature=1.0, top_k=40),
    "top-p 0.9": dict(temperature=1.0, top_p=0.9),
    "min-p 0.1": dict(temperature=1.0, min_p=0.1),
}
```

Eight ways to continue one prompt with the Chapter 2.5 model

```
$ uv run -m u2.decode
prompt: 'Once upon a time, there was a little dog named'; 10 continuations of up to 120
    tokens each
strategy               log-prob/token  repeated 4-grams  distinct 3-grams
greedy                         -0.477              6.0%        one output
beam search, width 4           -0.298              0.0%        one output
temperature 0.7                -0.666              1.1%             82.3%
temperature 1.0                -1.222              1.3%             89.5%
temperature 1.5                -7.226              0.0%             99.1%
top-k 40                       -1.043              0.4%             89.6%
top-p 0.9                      -0.785              0.9%             86.1%
min-p 0.1                      -0.647              1.3%             81.1%
greedy: ...Spot. Spot loved to play with his ball. One day, Spot saw a big cat. The cat was
    very rude. Spot wanted to play with the cat, but the cat did not want to play. Spot had
    an idea. He would play with the cat and his ball. He would throw the ball and the cat
    would run to catch it. Spot was very happy. H
beam search, width 4: ...Spot. Spot loved to run and play all day. One day, Spot saw a big
    ball in the yard. He wanted to play with it, but it was too high for him to reach. Spot
    saw his friend, Cat, and asked, "Can you help me get the ball?" Cat said, "Yes, I can
    help you!" Cat climbed up the tree and got the ball for Spot
temperature 1.5: ...Bow pushed openatways case and a cared friend hiding in theBella's
    Nobody arrive. Finally, the mailed nothingicted out until it turned to call
    boardriedplasher permission before the check expectation. In a few moments smelledfic
    gonna<unk>lyAlright someone crashing!' Dodo names sirAliceEw chefs nose and
min-p 0.1: ...Spot. Spot loved to play and run around all day. One day, Spot saw a big,
    scary cat. The cat wanted to catch Spot. Spot knew he had to escape. He ran very fast
    to a tree. He hid behind the tree. The cat looked up and saw Spot. The cat was scared,
    but it had an idea. The cat jumped and caught Spot! T
```

The two deterministic methods find the most probable text, beam search more so, at −0.30 nats per token against greedy's −0.48, and both give one story per prompt however often they run. Greedy decoding already repeats 6% of its four-token sequences within 120 tokens. Beam search's story does not repeat here, and is fluent and generic. Sampling at temperature 1.0 keeps the model's own distribution and is far more varied, nine in ten three-token sequences distinct across ten samples, at a cost of three-quarters of a nat per token in probability. At 1.5 the tail dominates and the text breaks into fragments of words, at −7.2 nats per token. Truncation recovers probability at little cost in variety. Top-k 40 keeps all of pure sampling's variety at a fifth of a nat better probability, top-p 0.9 gives up a little variety for more probability, and min-p 0.1 lands next to temperature 0.7 on both measures, with stories that read as well as greedy's.

Judging quality needs a reader, or a model acting as one, as in Chapter 3.6.

PITFALL

Sampled text varies widely between runs. Judging a strategy by one or two samples, and especially by a sample chosen because it looked good, says little. Compare strategies on many samples with fixed seeds, and measure.

The KV cache

Without a cache, generating each new token runs the model over the whole sequence so far: the work for the t-th token grows with t, and the work for a sequence of n tokens with n^2. Most of that work repeats earlier work. With causal attention, the keys and values of earlier positions never change once computed, so they can be kept. Each step then computes the query, key and value of one new token per layer, appends the key and value to the cache, and attends over everything stored.

The KV cache: a pair of tensors per model, filled one step at a time — code/aif/gpt.py

```
class KVCache:
    def __init__(self, config: Config, batch: int, length: int, device, dtype=torch.float32):
        shape = (config.n_layers, batch, config.n_kv_heads, length, config.head_dim)
        self.k = torch.zeros(shape, device=device, dtype=dtype)
        self.v = torch.zeros(shape, device=device, dtype=dtype)
        self.length = 0

    def update(
        self, layer: int, k: torch.Tensor, v: torch.Tensor
    ) -> tuple[torch.Tensor, torch.Tensor]:
        end = self.length + k.shape[2]
        self.k[layer, :, :, self.length : end] = k
        self.v[layer, :, :, self.length : end] = v
        return self.k[layer, :, :, :end], self.v[layer, :, :, :end]
```

Chapter 2.4's attention uses the cache when one is passed, with a mask that lets the new tokens see every cached position. The program generates the longest sequence the model allows, with and without the cache, checks that the outputs are identical, and times each token:

The same generation, with and without the cache — code/u2/kvcache.py

```
with_cache, t_cache = timed_generation(model, prompt, new, True, d)
without, t_plain = timed_generation(model, prompt, new, False, d)
```

Greedy generation of 508 tokens, timed on the GPU and on the CPU

```
$ uv run -m u2.kvcache
greedy generation of 508 tokens; identical output with and without the cache
  mps  without the cache    4.79 s; first 50 tokens   5.2 ms each, last 50  14.0 ms
  mps  with the cache       3.60 s; first 50 tokens   5.6 ms each, last 50   6.4 ms
  cpu  without the cache   13.64 s; first 50 tokens   9.4 ms each, last 50  39.5 ms
  cpu  with the cache       2.46 s; first 50 tokens   4.2 ms each, last 50   4.7 ms
KV-cache size for one sequence:
  this model, 512 tokens, float32                           8.0 MiB
  SmolLM2-1.7B, 8,192 tokens, bfloat16                  1,536.0 MiB
  a 70B model with 8 KV heads, 128K tokens, bfloat16   40,960.0 MiB
```

Both ways produce the same 508 tokens. Without the cache, the cost of a token grows with its position: on the CPU from 9.4 ms for each of the first 50 tokens to 39.5 ms for each of the last 50, and on the GPU from 5.2 to 14.0 ms. With the cache the cost stays flat apart from noise: from 4.2 to 4.7 ms on the CPU, and between about 5 and 10 ms on the GPU, where the first and last 50 tokens average 5.6 and 6.4 ms (Figure 2.7.1). The cache makes the whole sequence 5.5 times faster on the CPU and only 1.3 times faster on the GPU. There a step of this 28M-parameter model costs mostly the fixed time of launching its kernels, and for the first two hundred tokens the GPU recomputes the whole sequence as quickly as it extends the cache by one token. Once the cache removes the arithmetic, the CPU generates faster than the GPU. Decoding one sequence a token at a time leaves a GPU nearly idle, which is why Chapter 4.8 batches many sequences together.

The cache costs memory. Each layer stores a key and a value vector per key-value head for every token:

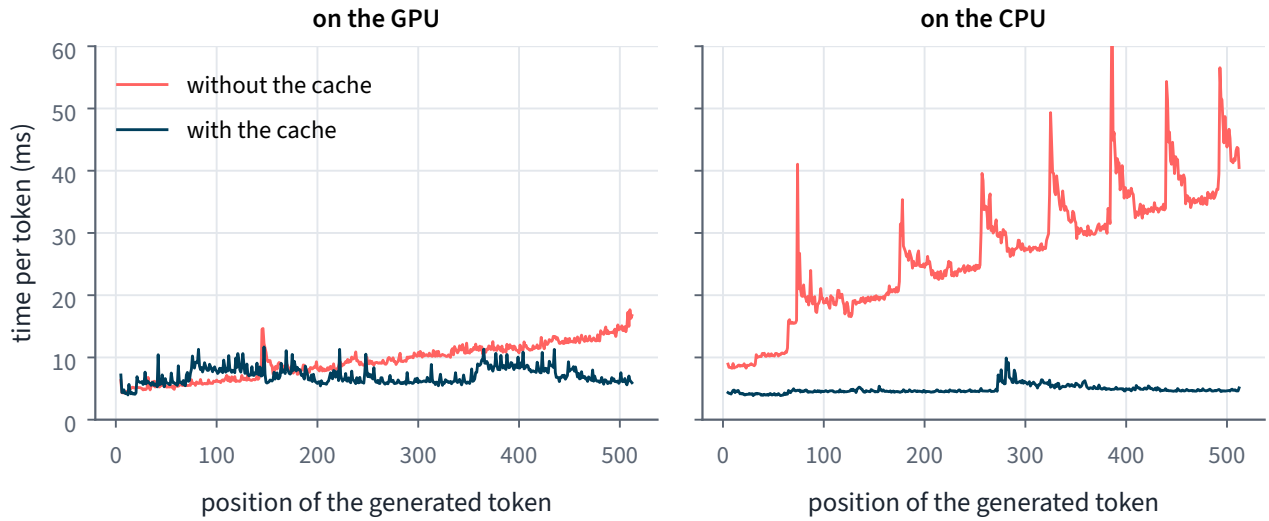


Figure 2.7.1. Time per generated token against its position, on each device. Without the cache, the cost of a token grows with the length of the sequence wherever arithmetic dominates. With it, the cost barely grows.

The cache's size `code/u2/kvcache.py`

```python
def cache_bytes(
    layers: int, kv_heads: int, head_dim: int, tokens: int, bytes_per_value: int
) -> int:
    return 2 * layers * kv_heads * head_dim * tokens * bytes_per_value
```

For the course's model at 512 tokens that is a few megabytes, but for large models at long contexts it rivals or exceeds the weights, and it grows with every sequence served at once. Chapter 2.4′s grouped-query attention shares key-value heads to reduce it, Chapter 4.7 quantises the cache, and Chapter 4.8 manages it in blocks.

KEY IDEA

The decoding rule changes the output as much as the weights do. Greedy choice and beam search find likely text and tend to loop. Pure sampling draws tokens from the tail. Sampling with truncation, min-p or top-p at a moderate temperature, keeps the variety and removes the tail. The KV cache makes each new token cost one token's work, at the price of memory that grows with the context.

IN THE LIBRARIES

Hugging Face's `generate` takes `do_sample`, `temperature`, `top_k`, `top_p`, `min_p` and `num_beams`, and keeps its cache in a `DynamicCache` object. `llama.cpp` chains samplers in a configurable order. `torch.multinomial` draws from a distribution, and a `torch.Generator` with a fixed seed makes the draws repeatable.

Reading

- Ari Holtzman and others, "The Curious Case of Neural Text Degeneration", ICLR 2020: nucleus sampling, and why likely text is dull.
- Minh Nhat Nguyen and others, "Turning Up the Heat: Min-p Sampling for Creative and Coherent LLM Outputs", ICLR 2025.
- Clara Meister, Ryan Cotterell and Tim Vieira, "If Beam Search is the Answer, What was the Question?", EMNLP 2020.
- Reiner Pope and others, "Efficiently Scaling Transformer Inference", MLSys 2023: the KV cache's memory and the cost of decoding.

Labs

LAB 2.7.1 A repetition penalty — 1.5 hours

Goal. Implement and measure the commonest fix for loops.

Code. `code/aif/sample.py`

1. Add a penalty that divides the positive logits, and multiplies the negative ones, of every token already in the output by a factor r.
2. Measure greedy decoding's repetition and log-probability for r = 1, 1.1, 1.3 and 1.5, and read the outputs.

DONE WHEN

- [ ] Your table has four rows.
- [ ] You can describe what goes wrong at the largest penalty, with an example.

EVIDENCE FOR THE NOTEBOOK

- The code, the table and the example.

LAB 2.7.2 Temperature and truncation — 1.5 hours

Goal. Measure what truncation gains at each temperature.

Code. `code/u2/decode.py`

1. Measure log-probability and distinct 3-grams at temperatures from 0.3 to 2.0, for pure sampling and for min-p 0.1.
2. Plot both measures against temperature for both strategies.

DONE WHEN

- [ ] Your plot has both strategies over the whole range.
- [ ] You can state the highest temperature at which each still writes readable stories, with samples to show it.

EVIDENCE FOR THE NOTEBOOK

- The plot and the samples.

LAB 2.7.3 The width of the beam — 1 hour

Goal. Find where wider search stops helping.

Code. `code/aif/sample.py`

1. Run beam search with widths 1, 2, 4, 8 and 16 on five prompts, and record log-probability per token, repetition and length.
2. Repeat with length penalties of 0 and 2.

DONE WHEN

- [ ] Your table covers every width and both penalties.
- [ ] You can explain how the penalty changes the outputs' lengths.

EVIDENCE FOR THE NOTEBOOK

- The table.

LAB 2.7.4 Many samples at once 1.5 hours

Goal. Measure what batching does for generation.

Code. `code/aif/sample.py`

1. Generate 16 continuations of one prompt with `generate_group`, which decodes them as one batch, and then with 16 calls of `generate`.
2. Time both, and repeat for 1, 4, 16 and 64 continuations.

DONE WHEN

- [ ] Your table has both times at four sizes.
- [ ] You can explain why the batch costs little more than one sequence, using Chapter 1.1's measurements of matrix products.

EVIDENCE FOR THE NOTEBOOK

- The table and the explanation.

Problem set

1. Show that sampling at temperature T approaches greedy choice as $T \to 0$, assuming one most likely token. What does it approach as $T \to \infty$?
2. A distribution has probabilities 0.5, 0.2, 0.12, 0.08, 0.06 and 0.04. Which tokens survive top-k with $k = 3$, top-p with $p = 0.75$, and min-p with 0.3?
3. SmolLM2-360M has 32 layers, 5 key-value heads and heads of 64 dimensions. How large is its KV cache for one sequence of 8,192 tokens in bfloat16? For 32 such sequences?
4. Generating n tokens without a cache, how do the total work of the feed-forward layers and of attention grow with n? With a cache?
5. Why does beam search with a wide beam tend to prefer short outputs, and how does dividing the score by length raised to a power correct it?

COMPLETION CHECKLIST

- [ ] I can implement greedy choice, beam search, temperature and top-k, top-p and min-p sampling.
- [ ] I can explain why likely text loops and sampled text wanders, and choose a strategy for a task.
- [ ] I can measure decoding strategies and avoid judging them by a few samples.
- [ ] I can implement a KV cache, compute its size, and explain what it saves.
- [ ] Lab 2.7.1–2.7.4 done-when criteria all hold, with evidence filed.

CHAPTER 2.8

Inside the model

TIME	7 hours
YOU BUILD	Tools that record every attention pattern and switch heads off; the induction heads in Chapter 2.5's model, found by their attention and confirmed by ablation; the logit lens, which reads the model's developing prediction after every layer; and a linear probe that finds the model tracking whether it is inside a quotation, against an untrained control.
NEEDS	Chapters 2.3, 2.4 and 2.5.

WHY IT MATTERS

A trained network's weights are millions of numbers set by training, and the model does things that nobody programmed explicitly. *Interpretability* asks what algorithms those weights implement. For transformers a few tools find real structure: attention patterns show where information moves, ablations test whether a component matters, the logit lens reads the prediction as it forms, and probes test whether a property is represented. This chapter uses them to find a specific mechanism in the course's model, confirm it causally, and find a feature it learned without labels. Researchers use the same methods, scaled up, to audit large models.

SKIP TEST

1. What is the residual stream, and why can every layer read from and write to it?
2. What does an induction head do, and why does it need two layers?
3. What does the logit lens assume, and when does that assumption fail?
4. What does a linear probe's accuracy show, and what does it not show?

If all four are easy, read the sections on the logit lens and on probing, and do Labs 2.8.2 and 2.8.4.

Core ideas

The residual stream

In Chapter 2.4's pre-norm transformer, each block adds its attention's output and then its feed-forward network's output to a running vector at each position, and never overwrites it. That vector, the *residual stream*, is a shared channel: every component reads the sum of everything written before it and adds its own contribution. Attention is the only component that moves information between positions, and the feed-forward networks transform it within a position. Reading the stream after each block shows what the model knows at each depth:

The residual stream after the embedding and after each block — code/aif/inspect.py

```
@torch.no_grad()
def residual_stream(model: GPT, idx: torch.Tensor) → list[torch.Tensor]:
    T = idx.shape[1]
    cos, sin = model.cos[:T], model.sin[:T]
    x = model.embed(idx)
    states = [x]
    for block in model.blocks:
        x = block(x, cos, sin)
        states.append(x)
    return states
```

To look at attention, and to switch individual heads off, the program replaces each attention layer's forward pass, temporarily, with one that computes the attention weights explicitly, keeps them, and scales each head's output by a mask:

Recording attention weights, with an optional mask over heads — code/aif/inspect.py

```
@contextlib.contextmanager
def recording(model: GPT, head_mask: torch.Tensor | None = None):
    """Attention layers keep their weights in .weights; head_mask scales each head."""

    def instrument(attn, layer: int):
        def forward(x, cos, sin, cache=None, layer_index=0):
            B, T, _ = x.shape
            q = attn.q(x).view(B, T, attn.n_heads, attn.head_dim).transpose(1, 2)
            k = attn.k(x).view(B, T, attn.n_kv_heads, attn.head_dim).transpose(1, 2)
            v = attn.v(x).view(B, T, attn.n_kv_heads, attn.head_dim).transpose(1, 2)
            q, k = apply_rope(q, cos, sin), apply_rope(k, cos, sin)
            group = attn.n_heads // attn.n_kv_heads
            k, v = k.repeat_interleave(group, dim=1), v.repeat_interleave(group, dim=1)
            scores = q @ k.transpose(-1, -2) / math.sqrt(attn.head_dim)
            causal = torch.ones(T, T, dtype=torch.bool, device=x.device).tril()
            weights = scores.masked_fill(~causal, float("-inf")).softmax(-1)
            attn.weights = weights.detach()
            y = weights @ v
            if head_mask is not None:
                y = y * head_mask[layer].view(1, -1, 1, 1)
            return attn.o(y.transpose(1, 2).reshape(B, T, -1))

        return forward

    for i, block in enumerate(model.blocks):
        block.attn.forward = instrument(block.attn, i)
    try:
        yield
    finally:
        for block in model.blocks:
            del block.attn.forward
```

Induction heads

Text repeats names, phrases and lines of dialogue. A model that has seen "Lily went to the park" earlier in a story should predict "park" after a later "Lily went to the". The mechanism that does this, found by Anthropic's interpretability researchers, is the *induction head*: at the current token A, it attends to the token that followed an earlier occurrence of A, and copies it. It needs two heads in different layers. A *previous-token head* in an earlier layer writes into each position the identity of the token before it. The induction head then searches with a query for "the current token" against keys for "the token before me", which only that earlier head's output makes available. Chapter 2.3's two-layer copying model learned a version of the same circuit.

The standard test uses meaningless text: 100 random tokens, then the same 100 again. A model with strong induction heads predicts the second copy almost perfectly. The course's model predicts the second copy barely better than the first, as the last line of the console below shows. Children's stories almost never repeat a long passage word for word, so copying arbitrary sequences was never worth learning. Stories do repeat names, so the program writes short stories that mention an invented name three times, a capital and two lower-case pieces that appear in no story, such as "Elgenals":

Stories that mention an invented name three times — code/u2/induction.py

```
TEMPLATES = [
    "Once upon a time, there was a little girl named {}. {} liked to play in the park. "
    "One day, {} saw a big red bird.",
    "There was a boy called {}. Every morning, {} ran to school. "
    "His friend waved at {} from the gate.",
    "A happy dog named {} lived on a farm. {} loved to dig in the mud. "
    "The farmer laughed at {} every day.",
    "In a small house lived a cat named {}. {} liked warm milk. "
```

```
    "When it rained, {} slept by the fire.",
]


def stories(tok, count: int, rng: np.random.Generator):
    """Stories that mention an invented name three times: a capital and two lower-case
pieces."""
    pieces = [tok.decode([i]) for i in range(tok.vocab_size)]
    capitals = [
        i for i, p in enumerate(pieces) if len(p) == 3 and p[1].isupper() and p[0] == " "
    ]
    lower = [
        i for i, p in enumerate(pieces) if 2 <= len(p) <= 4 and p.isalpha() and p.islower()
    ]
    rows, starts = [], []
    for n in range(count):
        name = [int(rng.choice(capitals)), int(rng.choice(lower)), int(rng.choice(lower))]
        parts = TEMPLATES[n % len(TEMPLATES)].split(" {}")
        ids, at = tok.encode(parts[0]), []
        for part in parts[1:]:
            at.append(len(ids))
            ids += name + tok.encode(part)
        rows.append(ids)
        starts.append(at)
    width = max(len(r) for r in rows)
    eot = tok.special["◁endoftext▷"]
    batch = torch.tensor([r + [eot] * (width - len(r)) for r in rows])
    return batch, torch.tensor(starts)
```

At its first mention, the name's second and third tokens are unpredictable. At the later mentions, a model that copies can predict them from the first token. Each head's *induction score* is its average attention, from the first token of the name's second mention, to the token that followed the first token of its first mention:

Attention to the token after the earlier occurrence, per head — code/u2/induction.py

```
with torch.no_grad(), inspect.recording(model):
    model(batch[:, :-1])
    rows = torch.arange(len(batch), device=dev)
    query, key = starts[:, 1], starts[:, 0] + 1
    scores = torch.stack(
        [b.attn.weights[rows, :, query, key].mean(dim=0) for b in model.blocks]
    )
```

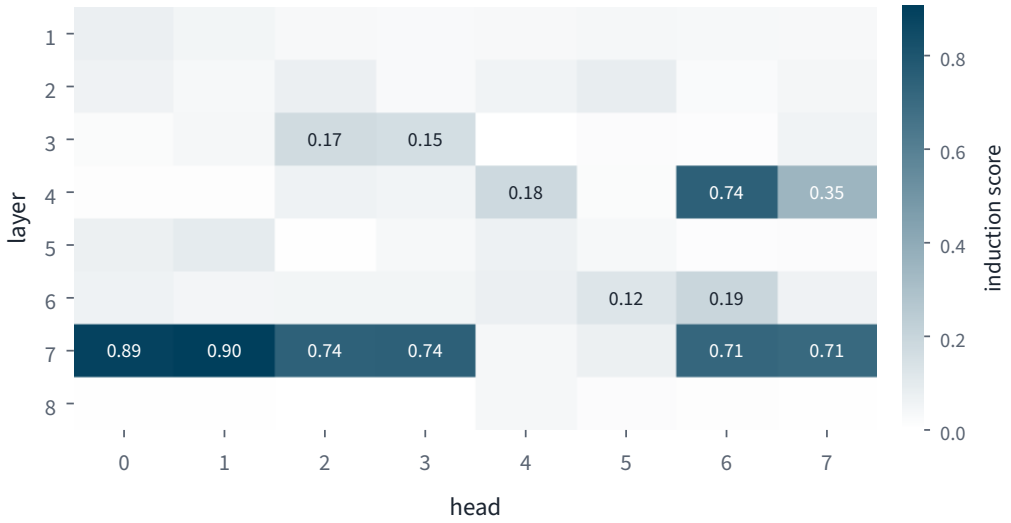


Figure 2.8.1. Induction scores of all 64 heads. Six heads of layer 7 and one of layer 4 put most of their attention on the position the mechanism predicts, and the other heads put little there.

Attention shows where information could flow, but it does not show that the model depends on that flow. An *ablation* tests dependence: switch the four highest-scoring heads off and measure the loss at the second mention again, then switch four other heads off instead, as a control.

Switching the induction heads off, and a control — code/u2/induction.py

```
mask = torch.ones(c.n_layers, c.n_heads, device=dev)
mask.view(-1)[top.indices] = 0
ablated = name_losses(model, batch, starts, mask)
others = torch.randperm(c.n_layers * c.n_heads, generator=torch.Generator().manual_seed(1))
others = [i for i in others.tolist() if i not in top.indices.tolist()][:4]
control = torch.ones(c.n_layers, c.n_heads, device=dev)
control.view(-1)[others] = 0
controlled = name_losses(model, batch, starts, control)
```

Induction in Chapter 2.5's model

```
$ uv run -m u2.induction
128 short stories, each naming an invented character three times, e.g.:
  'Once upon a time, there was a little girl named Elgenals. Elgenals liked to'
loss on the name's second and third tokens, at each mention: 11.58, 5.77, 5.47 nats
strongest induction heads, attention from the name's second mention to the token
that followed its first mention's first token:
  layer 7, head 1: 0.90
  layer 7, head 0: 0.89
  layer 7, head 3: 0.74
  layer 4, head 6: 0.74
second mention with those 4 heads switched off: 12.94 nats; with 4 other heads off: 5.68
for contrast, 100 random tokens repeated: 11.66 nats on the first copy, 11.31 on the second
```

At the first mention, the name's later tokens cost 11.6 nats, more than a uniform guess over the vocabulary, because the model expects ordinary words there. At the second and third mentions they cost 5.8 and 5.5. The four strongest heads, three of them in layer 7, put about three-quarters of their attention on the induction target. Switching them off raises the loss at the second mention to 12.9 nats, above the first mention's, so the copying is gone entirely, while switching off four other heads changes it by less than a tenth of a nat. A model trained on varied text applies this mechanism to any repeated sequence. In this model it is specialised to repeated names, the repetition its training data rewarded.

Induction heads are thought to underlie much of a model's *in-context learning*, its ability to use patterns from earlier in its prompt. In the models that Catherine Olsson and colleagues studied, they formed suddenly early in training, in a phase change visible as a bump in the loss curve, and Lab 2.8.2 looks for that moment in a model of your own.

The logit lens

The model's prediction is made by applying the final normalisation and the output layer to the residual stream after the last block. The *logit lens* applies the same two operations after every block, reading off what the model would predict if it stopped there:

The lens: the final norm and the output layer, applied early — code/u2/logit_lens.py

```
def lens(model, state: torch.Tensor) -> torch.Tensor:
    return model.head(model.norm(state))
```

The lens's loss after each block, on validation text code/u2/logit_lens.py

```
valid = np.memmap(data.DATA / "tinystories" / "valid-8192.bin", dtype=np.uint16, mode="r")
starts = np.linspace(0, len(valid) - 257, 64).astype(int)
batch = torch.from_numpy(
    np.stack([valid[s : s + 257] for s in starts]).astype(np.int64)
).to(dev)
losses = []
for state in inspect.residual_stream(model, batch[:, :-1]):
    with torch.no_grad():
        logits = lens(model, state)
    losses.append(
        F.cross_entropy(logits.flatten(0, 1).float(), batch[:, 1:].flatten()).item()
    )
```

The prediction forming, layer by layer

```
$ uv run -m u2.logit_lens
prompt: 'Once upon a time, there was a little girl named Lily. She loved to play with her'
top three next tokens read from the residual stream after each layer:
  embedding  ' her' 1.00, ' she' 0.00, ' hers' 0.00
  layer 1    ' mom' 0.43, ' friends' 0.34, ' friend' 0.08
  layer 2    ' mom' 0.51, ' friends' 0.19, ' friend' 0.12
  layer 3    ' mom' 0.55, ' friends' 0.14, ' friend' 0.10
  layer 4    ' mom' 0.64, ' friends' 0.10, ' friend' 0.05
  layer 5    ' friends' 0.45, ' mom' 0.15, ' family' 0.06
  layer 6    ' friends' 0.38, ' toy' 0.10, ' mom' 0.09
  layer 7    ' toy' 0.45, ' friends' 0.14, ' mom' 0.12
  layer 8    ' toys' 0.45, ' toy' 0.19, ' friends' 0.06
validation loss of the lens at each depth: 41.81, 5.79, 5.11, 4.36, 3.96, 3.26, 2.75, 1.79,
    1.30
```

The embedding alone predicts the current token, " her", as tied embeddings cause it to. After the first block the model already has a plausible continuation, " mom", which the early blocks grow more confident about. The middle blocks move to " friends", and the last two to a toy and then toys, the finished model's answer. On validation text the lens's loss falls with every block, from 5.8 nats after the first to 1.3 after the last, with the largest single drop in the seventh. Each block refines the prediction it receives instead of computing a new one, which the residual stream makes possible.

The lens assumes that every layer writes its intermediate guesses in the same coordinates as the last. In this small model that holds well enough to read. In larger ones the early layers read poorly, and the *tuned lens*, which learns a small correction for each layer, does better. Lab 2.8.3 builds one.

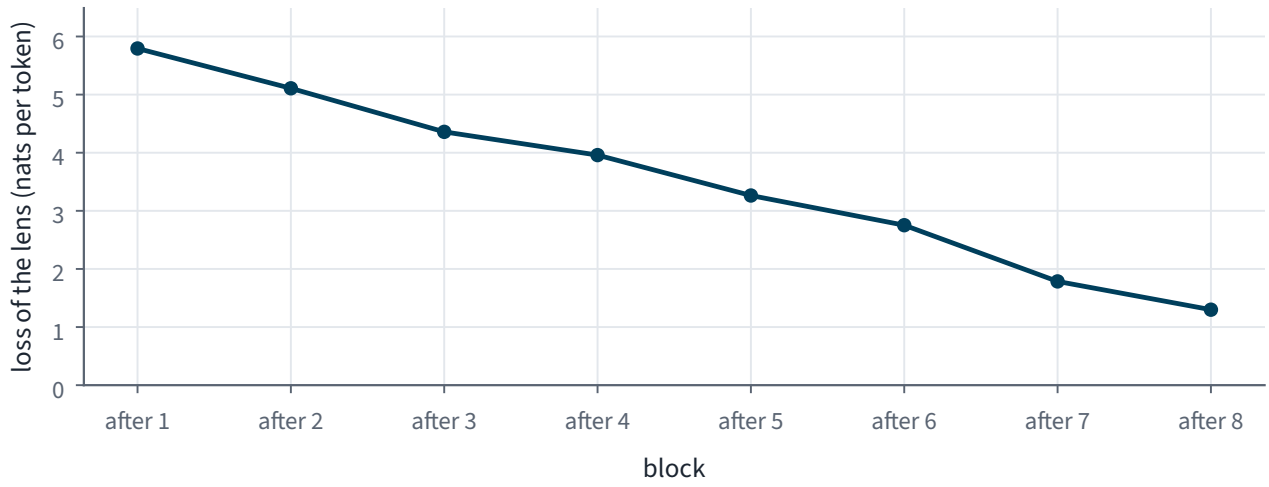


Figure 2.8.2. Validation loss of the logit lens after each block. The embedding alone is off the scale: with tied embeddings, its lens predicts the current token itself.

Probing for a feature

Dialogue changes what comes next in a story: inside quotation marks the characters speak in the first person, and the quotation must be closed. Nothing in training labelled dialogue, but a good model of stories should track it. A *linear probe* tests whether it does: label every position by whether it is inside a quotation, and train a linear classifier to predict the label from the residual stream at each depth.

Inside or outside a quotation, by counting quotation marks since the story began — code/u2/probe.py

```
def inside_quotes(tok, ids: list[int]) → np.ndarray:
    eot = tok.special["◁endoftext▷"]
    labels, quotes = [], 0
    for i in ids:
        if i == eot:
            quotes = 0
        labels.append(quotes % 2)
        quotes += tok.token_bytes[i].count(b'"')
    return np.array(labels)
```

A logistic-regression probe on standardised features — code/u2/probe.py

```
def probe_accuracy(train_x, train_y, test_x, test_y, steps: int = 300) → float:
    mean, std = train_x.mean(0), train_x.std(0) + 1e-6
    w = torch.zeros(train_x.shape[1], 2, device=train_x.device, requires_grad=True)
    b = torch.zeros(2, device=train_x.device, requires_grad=True)
    opt = torch.optim.Adam([w, b], lr=0.01)
    for _ in range(steps):
        loss = F.cross_entropy((train_x - mean) / std @ w + b, train_y)
        opt.zero_grad()
        loss.backward()
        opt.step()
    with torch.no_grad():
        return (((test_x - mean) / std @ w + b).argmax(1) == test_y).float().mean().item()
```

A probe needs controls. Only about one position in nine is inside a quotation, so a probe that always answers “outside” is right about 89% of the time. Some tokens are also far more common inside quotations, so even features that identify only the current token beat that base rate. The program therefore also probes a model with the same architecture and random weights.

Probe accuracy at each depth, for the trained model and an untrained one

```
$ uv run -m u2.probe
102,400 token positions, 10.9% of them inside quotation marks
depth        trained model  untrained model
embedding            91.9%            91.1%
layer 1              94.4%            90.9%
layer 2              95.4%            90.8%
layer 3              98.1%            90.4%
layer 4              98.0%            90.0%
layer 5              98.9%            89.7%
layer 6              99.4%            89.6%
layer 7              99.1%            89.3%
layer 8              98.7%            89.1%
```

Both probes on the embedding beat the base rate of 89%, because some tokens occur almost only inside quotations: the punctuation that closes a quotation, and first- and second-person words such as “I”, “me” and “you”. Above the embedding, the two models differ. The untrained model’s probe falls slightly with depth, as random layers blur each token’s identity, while the trained model’s rises to 99.4% after layer 6 and stays close to it. Knowing whether a quotation is open requires counting quotation marks back to the start of the story, so the trained model computes that state and keeps it, in a form a linear readout can find, through its middle and later layers.

A probe shows that information can be read linearly from the stream. It does not show that the model uses it. Showing use needs an intervention, such as changing the stream along the probe’s direction and watching the model’s behaviour change, which Lab 2.8.4 attempts.

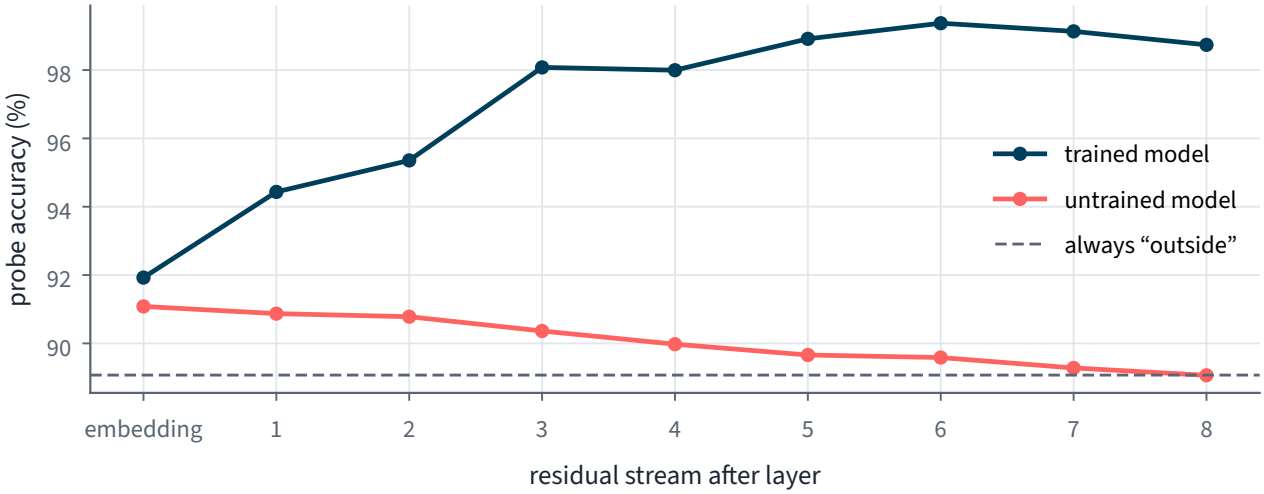


Figure 2.8.3. Probe accuracy against depth for the trained and untrained models, with the base rate of always answering “outside”.

PITFALL

Interpretations need controls. Compare a probe on a trained model with the base rate and with the same probe on an untrained model; compare an ablation with ablating random components of the same size. Without them, a probe may only be detecting token identity, and an ablation may only be measuring general damage.

KEY IDEA

The residual stream is a shared channel that every layer reads and writes. Attention patterns show where information moves, ablations test whether a component matters, the logit lens shows the prediction forming, and probes find what is linearly represented. With controls, they found two mechanisms in the course’s model that nobody programmed: a two-head copying circuit and a tracker of dialogue.

IN THE LIBRARIES

`register_forward_hook` records activations without replacing any code. This chapter replaces `forward` because it changes attention itself. TransformerLens provides named hooks on every activation of many open models, and nnsight does the same for any PyTorch model. Sparse autoencoders, trained on the residual stream to decompose it into many interpretable features, are the main current method for going beyond single probes.

Reading

- Nelson Elhage and others, “A Mathematical Framework for Transformer Circuits”, Transformer Circuits Thread, 2021.
- Catherine Olsson and others, “In-context Learning and Induction Heads”, Transformer Circuits Thread, 2022.
- nostalgebraist, “Interpreting GPT: the logit lens”, 2020; and Nora Belrose and others, “Eliciting Latent Predictions from Transformers with the Tuned Lens”, 2023.
- Guillaume Alain and Yoshua Bengio, “Understanding Intermediate Layers Using Linear Classifier Probes”, 2016.
- Adly Templeton and others, “Scaling Monosemanticity: Extracting Interpretable Features from Claude 3 Sonnet”, Transformer Circuits Thread, 2024.

Labs

LAB 2.8.1 Previous-token heads — 1.5 hours

Goal. Find the other half of the circuit.

Code. `code/u2/induction.py`

1. Score every head by its average attention to the immediately preceding position, on the program's stories.
2. Switch off the strongest previous-token heads and measure the loss at the name's second mention.

DONE WHEN

- [ ] You can name the previous-token heads and their layers, and they lie below the induction heads.
- [ ] You have the ablation's effect, compared with a control.

EVIDENCE FOR THE NOTEBOOK

- The scores and the losses.

LAB 2.8.2 When they form — 2 hours

Goal. Find when induction heads form during training.

Code. `code/u2/pretrain.py`

1. Train the small preset on 20 million tokens, saving a copy of the model every 100 steps; change the checkpoint's file name to include the step so that copies are kept.
2. For each copy, measure the loss at the second mention of the invented names and the best induction score, and plot both with the training loss.

DONE WHEN

- [ ] Your plot shows the three curves over training.
- [ ] You can name the step at which induction appears, and say whether the training loss shows a bump there.

EVIDENCE FOR THE NOTEBOOK

- The plot.

LAB 2.8.3 A tuned lens — 2 hours

Goal. Read the predictions of early layers more accurately.

Code. `code/u2/logit_lens.py`

1. For each layer, train a $d \times d$ matrix, starting from the identity, applied to the residual stream before the lens, to minimise the KL divergence from the model's final distribution on training windows.
2. Compare the tuned lens's validation loss at each depth with the logit lens's.

DONE WHEN

- [ ] You have both curves.
- [ ] You can say at which depths the tuned lens helps most, and why the last layer gains nothing.

EVIDENCE FOR THE NOTEBOOK

- The plot.

LAB 2.8.4 Steering by a direction 1.5 hours

Goal. Test whether the dialogue feature is used.

Code. `code/u2/probe.py`

1. Take the difference between the mean residual stream inside and outside quotations at the layer where the probe is best.
2. Add a multiple of it to the stream at that layer during generation, at three strengths, and read the stories.

DONE WHEN

- [ ] You have samples at each strength.
- [ ] You can say whether adding the direction makes the model open or continue dialogue, and what that shows that the probe could not.

EVIDENCE FOR THE NOTEBOOK

- The samples and the conclusion.

Problem set

1. Why can a model with one layer of attention not implement an induction head, using only the token and position information available to its keys?
2. The invented names' second and third tokens are drawn uniformly from about 1,100 lower-case pieces. What loss does a model that knows only that have on them at the first mention, and what does a perfect copier have at the second? Why does the course's model do worse than the first at the first mention?
3. With tied embeddings, the logit lens on the embedding layer predicts the current token. Show why, from the dot product between a token's embedding and every row of the output layer.
4. The probe's labels are 11% "inside". What accuracy does always answering "outside" achieve, and why can an untrained model's probe do better?
5. Ablating a head raises a loss by less than the head's role might suggest. Give two reasons why ablation can understate a component's importance.

COMPLETION CHECKLIST

- [ ] I can explain the residual stream, and read it after every block.
- [ ] I can find heads by their attention patterns and test them by ablation, with controls.
- [ ] I can explain how two heads compose into an induction head.
- [ ] I can apply the logit lens and a linear probe, and say what each shows and does not show.
- [ ] Lab 2.8.1–2.8.4 done-when criteria all hold, with evidence filed.

UNIT 2

Sign-off

A unit is complete when every chapter's completion checklist is ticked, its evidence is in your notebook and repository, and you can do each review task below without notes. Date each line as you finish it. If someone who trains models for a living will review your work, ask them to sign the last line.

	Chapter	Checklist ticked	Evidence filed	Date
2.1	Tokenisation	☐	☐	
2.2	Next-token prediction	☐	☐	
2.3	Attention	☐	☐	
2.4	The transformer	☐	☐	
2.5	Pretraining	☐	☐	
2.6	Scaling laws	☐	☐	
2.7	Decoding	☐	☐	
2.8	Inside the model	☐	☐	

UNIT REVIEW

Each task takes an hour or less if the unit has done its work. Do them aloud or in writing, from memory, then check against your notebook and code.

- ☐ Train a BPE tokeniser by hand on a ten-word corpus, and explain why a tokeniser trained on stories compresses code poorly.
- ☐ Derive attention from the problem of looking up a value by a fuzzy key, and explain why the scores are divided by $\sqrt{d}$.
- ☐ Draw your transformer's block from memory, with the shape of every tensor, and count its parameters and FLOPs per token.
- ☐ Explain, with your Chapter 2.6 fit, how you would split a fixed compute budget between model size and data.
- ☐ Explain what an induction head does, how you found one, and what switching it off did to the loss.

Signed: student _____ Date _____

Reviewed by (optional) _____ Date _____